Porto. TechHub

P.PORTO

AI-POWERED SOFTWARE ENGINEERING

| CONCEÇÃO, ARQUITETURA & PROTOTIPAGEM INTELIGENTE |

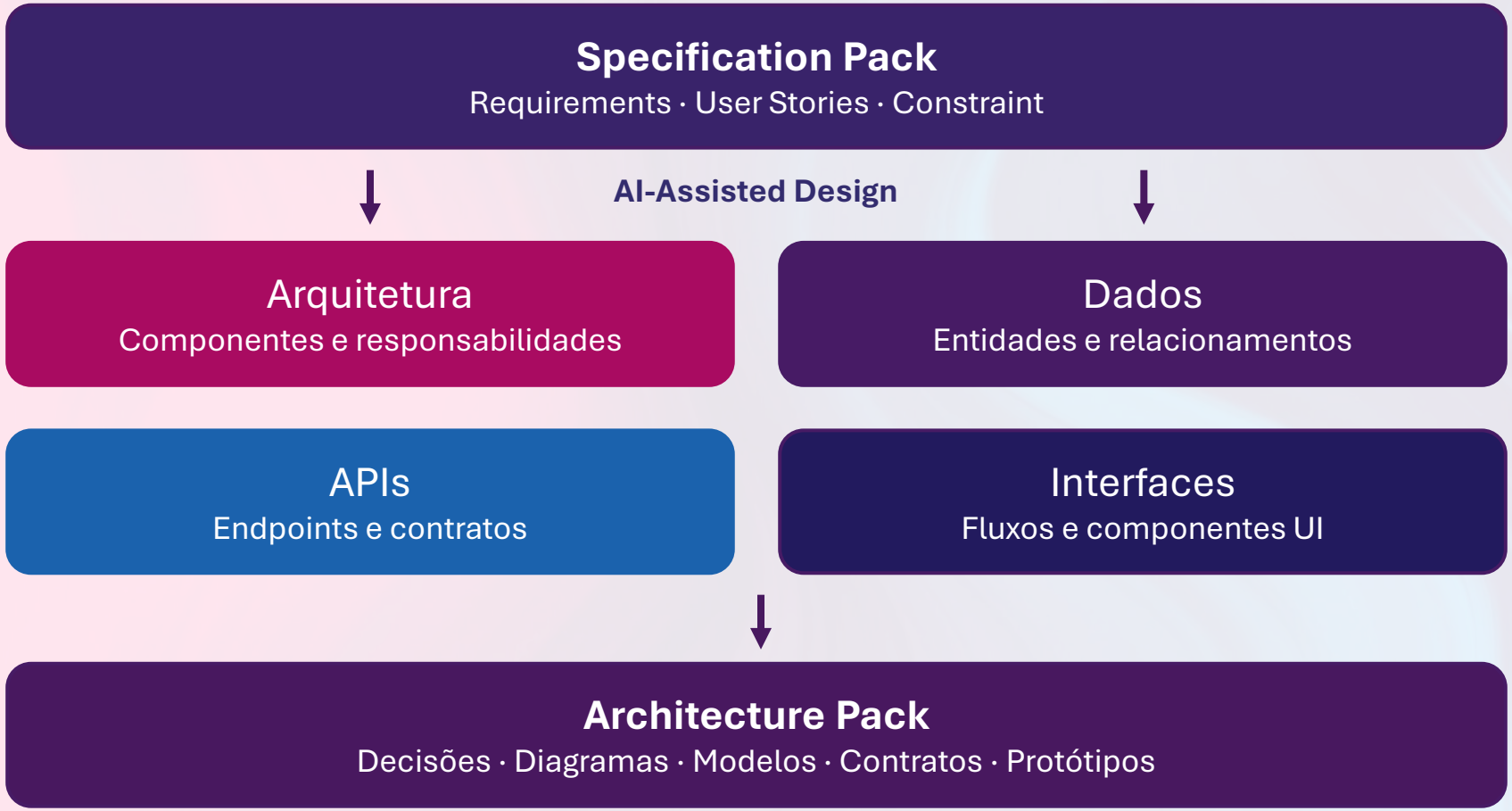
VITOR MANUEL SANTOS

ÍNDICE

- Da especificação à arquitetura de software
- IA como apoio à conceção de soluções
- Estilos arquiteturais e trade-offs
- Clean Architecture, Hexagonal Architecture e separação de responsabilidades
- Architecture Decision Records (ADR)
- Modelação de dados assistida por IA
- Diagramas ER, SQL e migrations
- API Design e contratos OpenAPI
- Prototipagem de interfaces com IA
- Validação arquitetural, segurança e boas práticas

DA ESPECIFICAÇÃO À ARQUITETURA

Transformar o **O quê?** no **Como?**



IA COMO SOFTWARE ARCHITECT?

- A IA não substitui a decisão arquitetural.

IA	Software Engineer
Propõe arquiteturas	Define critérios
Explora alternativas	Analisa trade-offs
Identifica padrões	Avalia adequação
Gera diagramas	Valida consistência
Identifica riscos	Assume decisões
Documenta decisões	Aprova a arquitetura

- AI as a Sparring Partner, not a Decision Maker.

EXPLORAR ALTERNATIVAS ARQUITETURAIS COM IA

- A mesma aplicação pode ter diferentes arquiteturas.

  Monolith

  Uma única aplicação, implantação simples e menor complexidade operacional.

  Modular Monolith

  Aplicação única com módulos bem definidos e responsabilidades isoladas.

  Microservices

  Serviços independentes, escalabilidade individual e maior complexidade distribuída.

- Critérios de comparação:
  - Complexidade · Escalabilidade · Manutenção · Deploy · Custos · Autonomia das equipas
- Questão central: Qual é a arquitetura mais adequada ao problema e ao contexto, e não apenas a mais sofisticada ou a mais recente?

PRINCÍPIOS DE ARQUITETURA DE SOFTWARE

- **Uma boa arquitetura não é necessariamente a mais complexa.**
- Princípios fundamentais:
  - Separation of Concerns: responsabilidades bem definidas.
  - Low Coupling: minimizar dependências entre componentes.
  - High Cohesion: agrupar funcionalidades relacionadas.
  - SOLID: facilitar manutenção e evolução.
  - KISS: evitar complexidade desnecessária.
  - YAGNI: não implementar funcionalidades antecipadamente.
- Com IA: Pedir uma análise crítica da arquitetura, identificando violações destes princípios e possíveis melhorias.

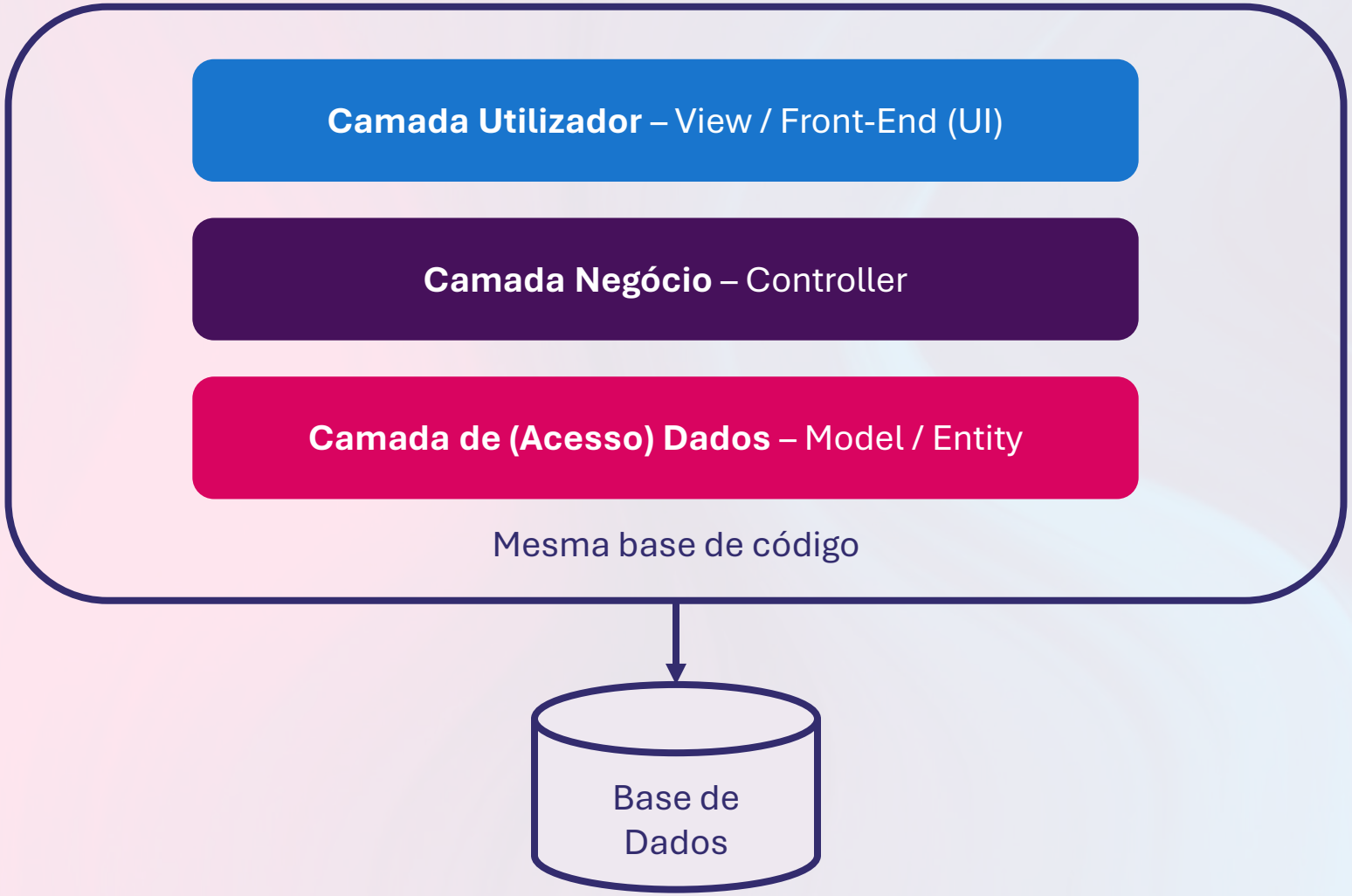
DIFERENTES ARQUITETURAS, DIFERENTES PROPÓSITOS

VITOR MANUEL SANTOS

MONÓLITO TRADICIONAL

VITOR MANUEL SANTOS

MONÓLITO TRADICIONAL



MONÓLITO TRADICIONAL

- Num monólito tradicional, toda a aplicação é desenvolvida e executada como uma única unidade.
- Normalmente, isso significa:
  - Uma única base de código principal
  - Um único processo de execução
  - Um único deployment
  - Forte acoplamento entre componentes
  - Partilha da mesma base de dados e do mesmo ciclo de entrega

MONÓLITO TRADICIONAL

- Sinais típicos desta abordagem:
  - Módulos e funcionalidades vivem todos no mesmo sistema
  - Alterações em diferentes áreas geram novo deployment do todo
  - Dependências internas tendem a crescer com o tempo
  - Fronteiras entre responsabilidades nem sempre são claras
  - Equipas trabalham sobre a mesma aplicação e o mesmo ciclo de entrega

MONÓLITO TRADICIONAL – EXEMPLO

- Uma plataforma interna onde:
  - Pedidos
  - Faturação
  - Notificações
  - Autenticação
  - Relatórios
  - Auditoria...

  estão todos implementados dentro da mesma aplicação.

MONÓLITO TRADICIONAL – VANTAGENS

- O monólito tradicional pode ser vantajoso quando:
  - A equipa é pequena ou média
  - O domínio ainda está a mudar muito
  - A operação precisa de ser simples
  - O orçamento é limitado
  - O sistema ainda não justifica complexidade distribuída

MONÓLITO TRADICIONAL – VANTAGENS

- Vantagens concretas:
  - Setup inicial mais simples
  - Deployment mais direto
  - Menor overhead operacional
  - Debugging potencialmente mais simples no arranque
  - Menor complexidade de infraestrutura

MONÓLITO TRADICIONAL – LIMITAÇÕES E RISCOS

- À medida que o sistema cresce, surgem problemas frequentes:
  - maior dificuldade em perceber impactos de alteração
  - acoplamento excessivo entre áreas do sistema
  - risco de regressões em mudanças pequenas
  - build, teste e deployment mais pesados
  - dificuldade em escalar partes específicas de forma independente
  - menor autonomia entre equipas

MONÓLITO TRADICIONAL – LIMITAÇÕES E RISCOS

- O problema não é ser monólito.
- O problema é o monólito tornar-se grande, confuso e difícil de evoluir.
- Um monólito tradicional pode ser adequado no início, mas pode tornar-se um entrave quando o contexto muda.

MONÓLITO TRADICIONAL – QUANDO FAZ SENTIDO

- O monólito tradicional pode ser uma escolha válida quando:
  - o produto ainda está numa fase inicial
  - a equipa precisa de entregar rápido
  - o domínio ainda não está suficientemente estabilizado
  - não existe necessidade real de escalabilidade seletiva
  - a prioridade é reduzir custo e complexidade operacional

MONÓLITO TRADICIONAL – EXEMPLO DE IMPLEMENTAÇÃO

- Num sistema como o **Matriz**:
  - ProjectController
  - TaskController
  - UserController
  - TeamController

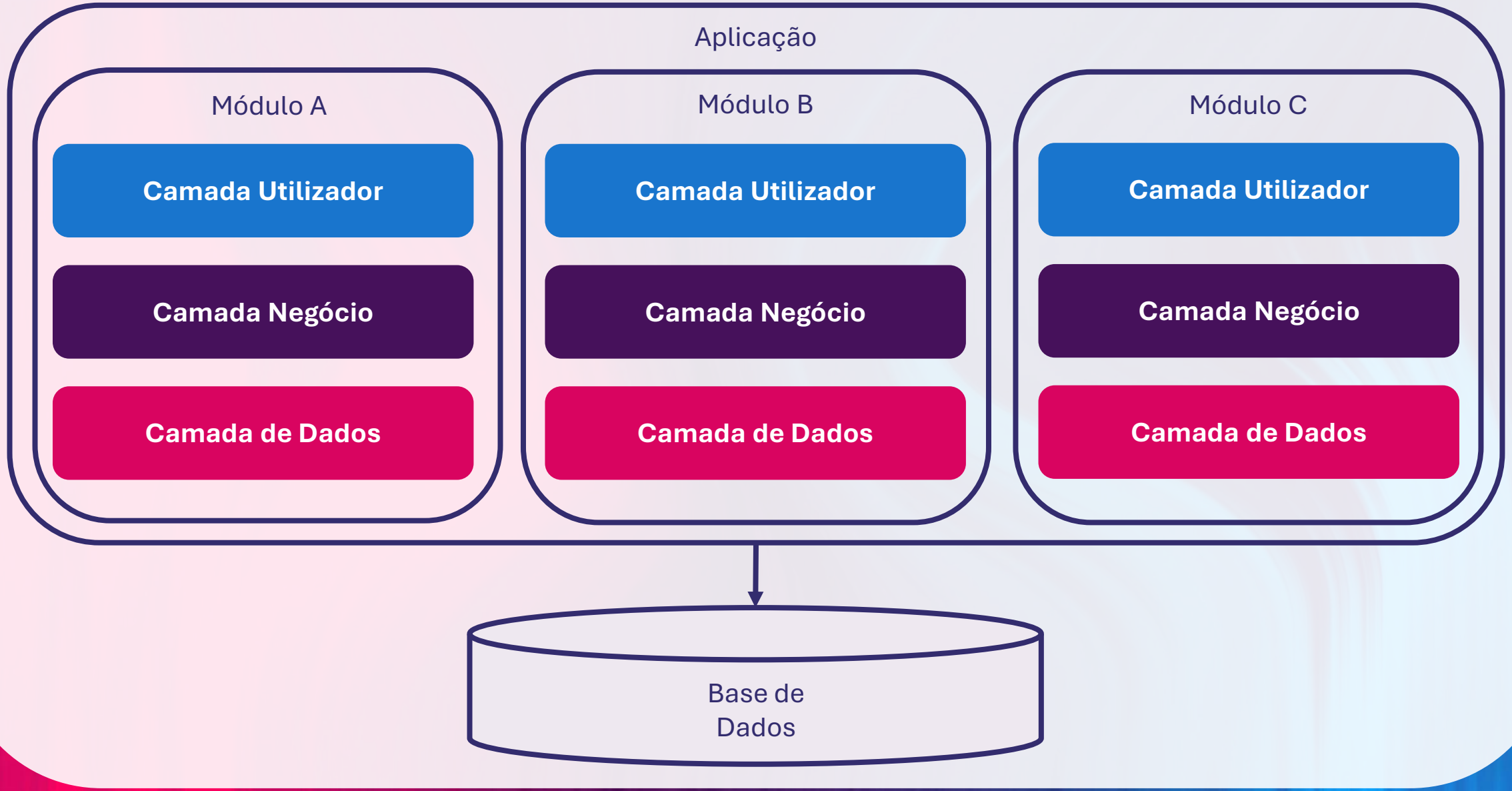
MONÓLITO TRADICIONAL – EXEMPLO DE IMPLEMENTAÇÃO

- Até aqui parece normal. O problema do monólito tradicional não é existirem estes controllers, é que muitas vezes eles:
  - acedem diretamente a várias áreas do sistema
  - chamam serviços de vários domínios
  - fazem validação, regras de negócio e acesso a dados no mesmo fluxo
  - conhecem demasiadas tabelas e demasiadas dependências

MONÓLITO MODULAR

VITOR MANUEL SANTOS

MONÓLITO MODULAR



MONÓLITO MODULAR

- Num monólito modular, o sistema é entregue como uma única aplicação, mas organizado internamente em módulos bem definidos.
- Exemplo de módulos:
  - Pedidos
  - Faturação
  - Clientes
  - Notificações
  - Auditoria

MONÓLITO MODULAR

- Características:
  - um único deploy
  - uma base de código principal
  - comunicação interna por chamadas locais
  - mais simples de desenvolver, testar e operar
  - exige disciplina para manter fronteiras entre módulos

MONÓLITO MODULAR – VANTAGENS

- **Melhor organização interna**: O sistema continua a ser uma aplicação única, mas com fronteiras mais claras entre módulos.
- **Evolução e manutenção mais fáceis**: É mais simples alterar uma área sem espalhar impacto por todo o sistema.
- **Menor complexidade operacional do que microsserviços**: Mantém deployment e operação mais simples, evitando o overhead de uma arquitetura distribuída.

MONÓLITO MODULAR – DESVANTAGENS

- **Continua a haver um único deployment**: Mesmo com boa separação interna, a aplicação é entregue como um todo.
- **Escalabilidade menos flexível**: Não permite escalar módulos específicos com a mesma liberdade que microsserviços.
- **Pode degradar-se com o tempo**: Se a disciplina arquitetural falhar, os módulos começam a misturar responsabilidades e o sistema aproxima-se de um monólito “confuso”.

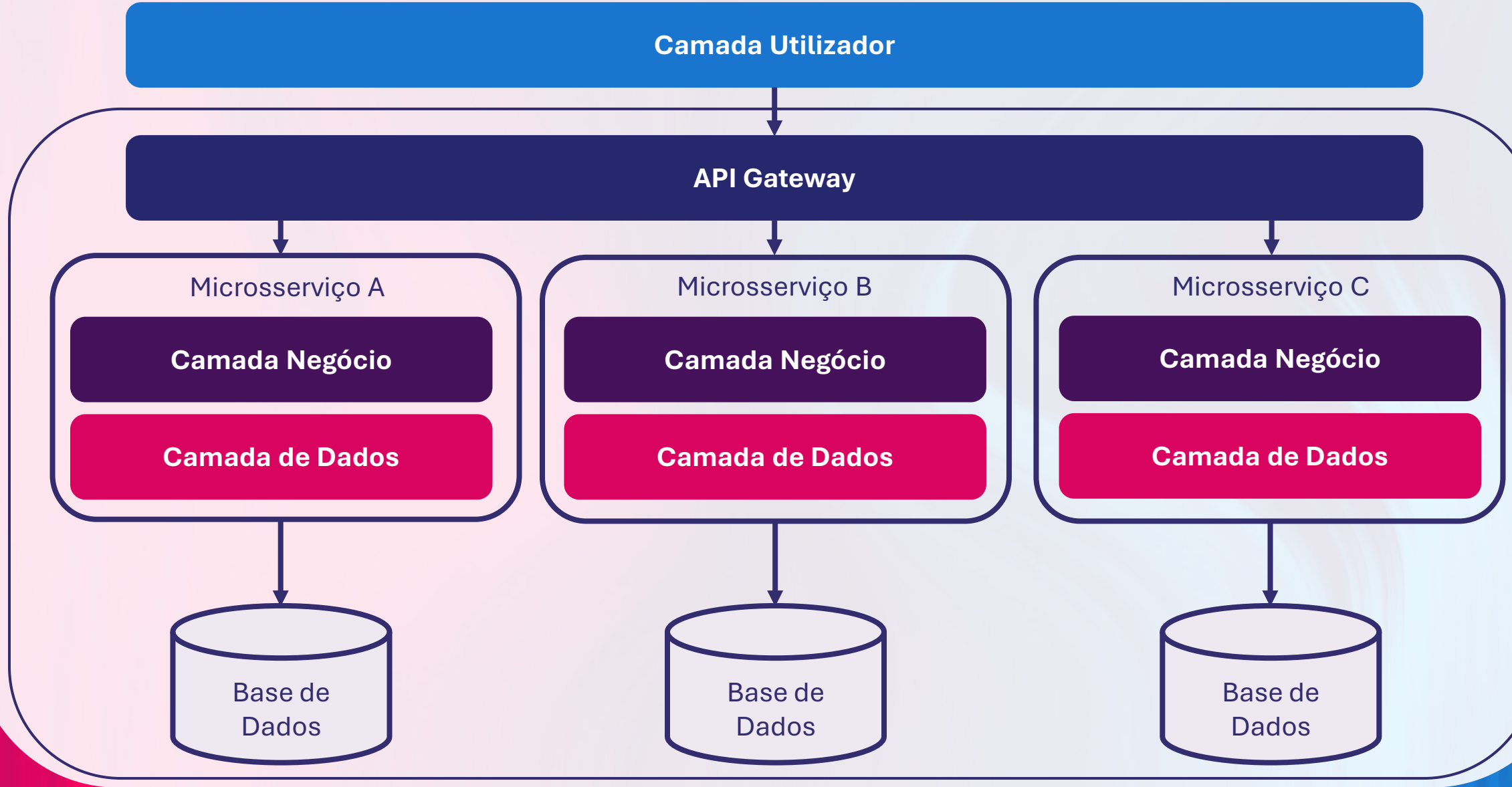
MONÓLITO MODULAR – EXEMPLO DE IMPLEMENTAÇÃO

- Num sistema como a Matriz:
  - ProjectController
  - ProjectService
  - ProjectRepository
- noutro módulo:
  - TaskController
  - TaskService
  - TaskRepository
- Tudo dentro da mesma aplicação.

MICROSSERVIÇOS

VITOR MANUEL SANTOS

MICROSSERVIÇOS



MICROSSERVIÇOS

- Em microsserviços, o sistema é dividido em vários serviços independentes, cada um responsável por uma capacidade de negócio bem delimitada.
- Exemplos:
  - serviço de pedidos
  - serviço de stock
  - serviço de faturação
  - serviço de notificações

MICROSSERVIÇOS

- Características:
  - cada serviço pode ter deploy próprio
  - comunicação entre serviços por rede, normalmente via API
  - API = Application Programming Interface, interface de comunicação entre sistemas
  - equipas podem evoluir serviços de forma mais autónoma
  - escalabilidade pode ser feita apenas onde é necessário

MICROSSERVIÇOS – VANTAGENS

- **Escalabilidade mais flexível**: Permitem escalar partes específicas do sistema de forma independente.
- **Maior desacoplamento entre domínios**: Cada serviço tende a ter responsabilidades mais focadas e fronteiras mais claras.
- **Deploys independentes**: É possível evoluir e publicar serviços separados sem obrigar ao deployment do sistema inteiro.

MICROSSERVIÇOS – DESVANTAGENS

- **Maior complexidade operacional**: Exigem mais esforço ao nível de infraestrutura, monitorização, comunicação e gestão de falhas.
- **Diagnóstico e manutenção mais difíceis**: Problemas distribuídos tendem a ser mais difíceis de localizar e analisar.
- **Overhead arquitetural mais elevado**: Introduzem mais decisões técnicas, contratos, dependências e mecanismos de coordenação.

MICROSSERVIÇOS – EXEMPLO DE IMPLEMENTAÇÃO

- Num sistema como a **Matriz**:
  - Serviço de Projetos
  - Serviço de Equipas
  - Serviço de Tarefas
- Cada um com:
  - Controller
  - Service
  - Repository
  - Modelo interno
- Em microsserviços, a regra saudável é não partilhar a mesma base de dados entre serviços.

MICROSSERVIÇOS – EXEMPLO DE IMPLEMENTAÇÃO

- Se os microsserviços não partilham a mesma base de dados, como funcionam as chaves?
  - Temos modulo de pagamentos com BD de pagamentos
  - Temos modulo de clientes com a BD de clientes.
- Como referenciamos que aquele cliente fez aquele pagamento?

- A relação é feita ao nível da aplicação e das mensagens, não ao nível relacional da base de dados.
- Em microsserviços, a relação deixa de ser relacional na base de dados, e passa a ser lógica entre bounded contexts / serviços.

- Perdemos a integridade referencial automática do SGBD entre serviços. Deixamos de ter o conforto de uma FK que impede incoerências diretamente na BD.

MONÓLITO MODULAR VS MICROSSERVIÇOS

VITOR MANUEL SANTOS

MONÓLITO MODULAR VS MICROSSERVIÇOS

- Monólito modular tende a ser mais adequado quando:
  - a equipa é pequena ou média
  - o produto ainda está a estabilizar
  - a simplicidade operacional é prioridade
  - o custo de operação precisa de ser controlado
  - a maioria das funcionalidades evolui em conjunto

MONÓLITO MODULAR VS MICROSSERVIÇOS

- Microsserviços tendem a fazer mais sentido quando:
  - existem domínios bem separados
  - há equipas autónomas por área
  - há necessidade de escalabilidade seletiva
  - diferentes partes do sistema têm ritmos de mudança distintos
  - o desacoplamento organizacional e técnico é crítico

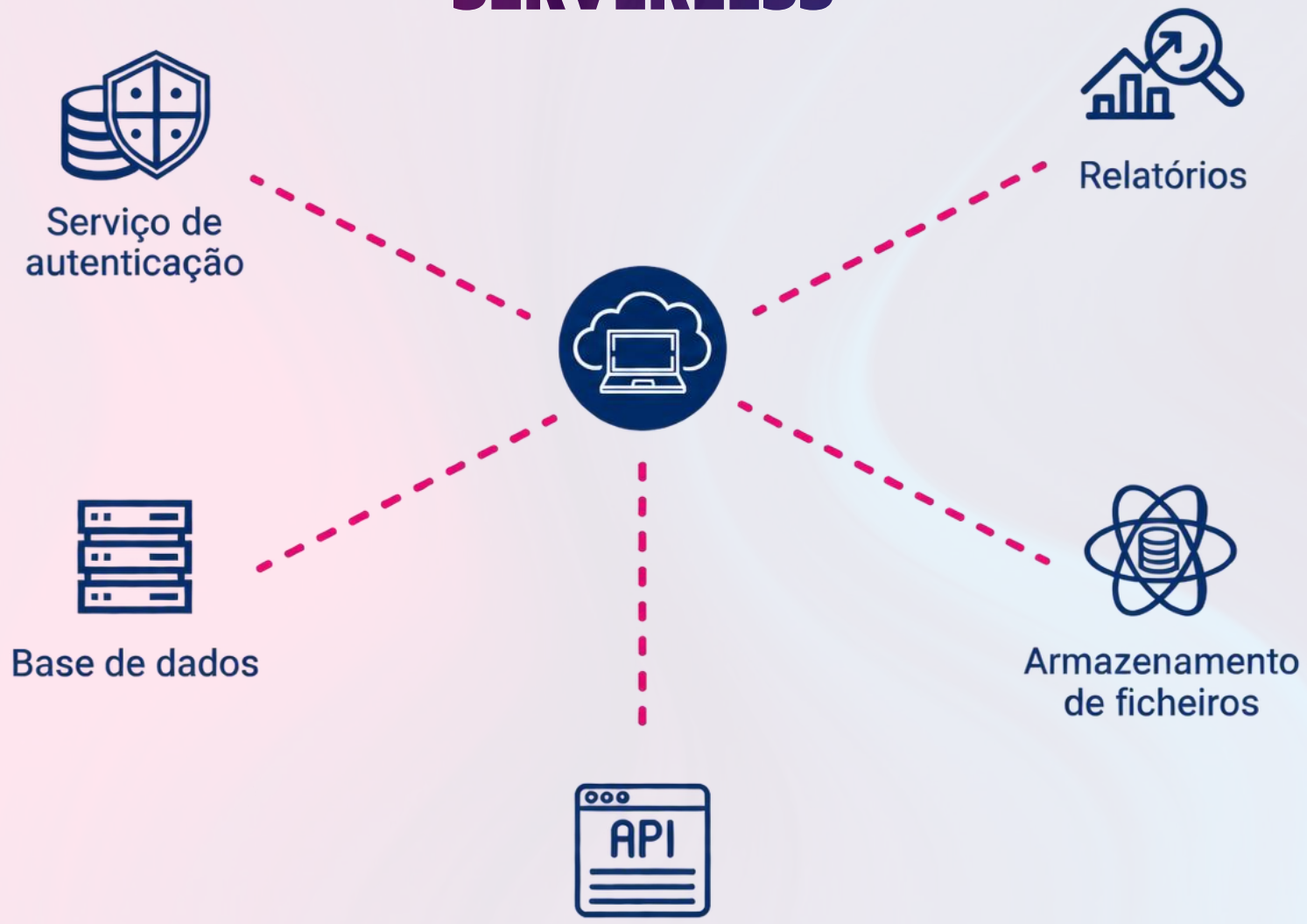
SERVERLESS

VITOR MANUEL SANTOS

SERVERLESS

- **Serverless** não significa que não existam servidores.
- Significa que a gestão desses servidores deixa de ser responsabilidade direta da equipa.

SERVERLESS


Serviço de autenticação
Relatórios
Base de dados
Armazenamento de ficheiros
API

SERVERLESS

- Numa abordagem **serverless**, a equipa desenvolve lógica de aplicação que é executada numa infraestrutura gerida pelo fornecedor cloud.
- Isto significa que:
  - não é necessário gerir servidores diretamente
  - o aprovisionamento e a escalabilidade são automáticos
  - a execução pode ser ativada por pedidos ou eventos
  - o custo tende a estar mais ligado ao uso real

SERVERLESS – VANTAGENS

- menor esforço de operação de infraestrutura
- escalabilidade automática
- arranque rápido para funcionalidades isoladas
- pagamento geralmente alinhado com a utilização
- boa integração com eventos e serviços cloud

SERVERLESS – RISCOS

- Maior dependência do fornecedor cloud
- Debugging e observabilidade podem tornar-se mais difíceis
- Cold starts em alguns cenários
- Gestão de estado mais delicada
- Arquitetura pode fragmentar-se demasiado
- Custo pode deixar de ser vantajoso em cargas muito constantes

SERVERLESS – RISCOS DE DESENHO

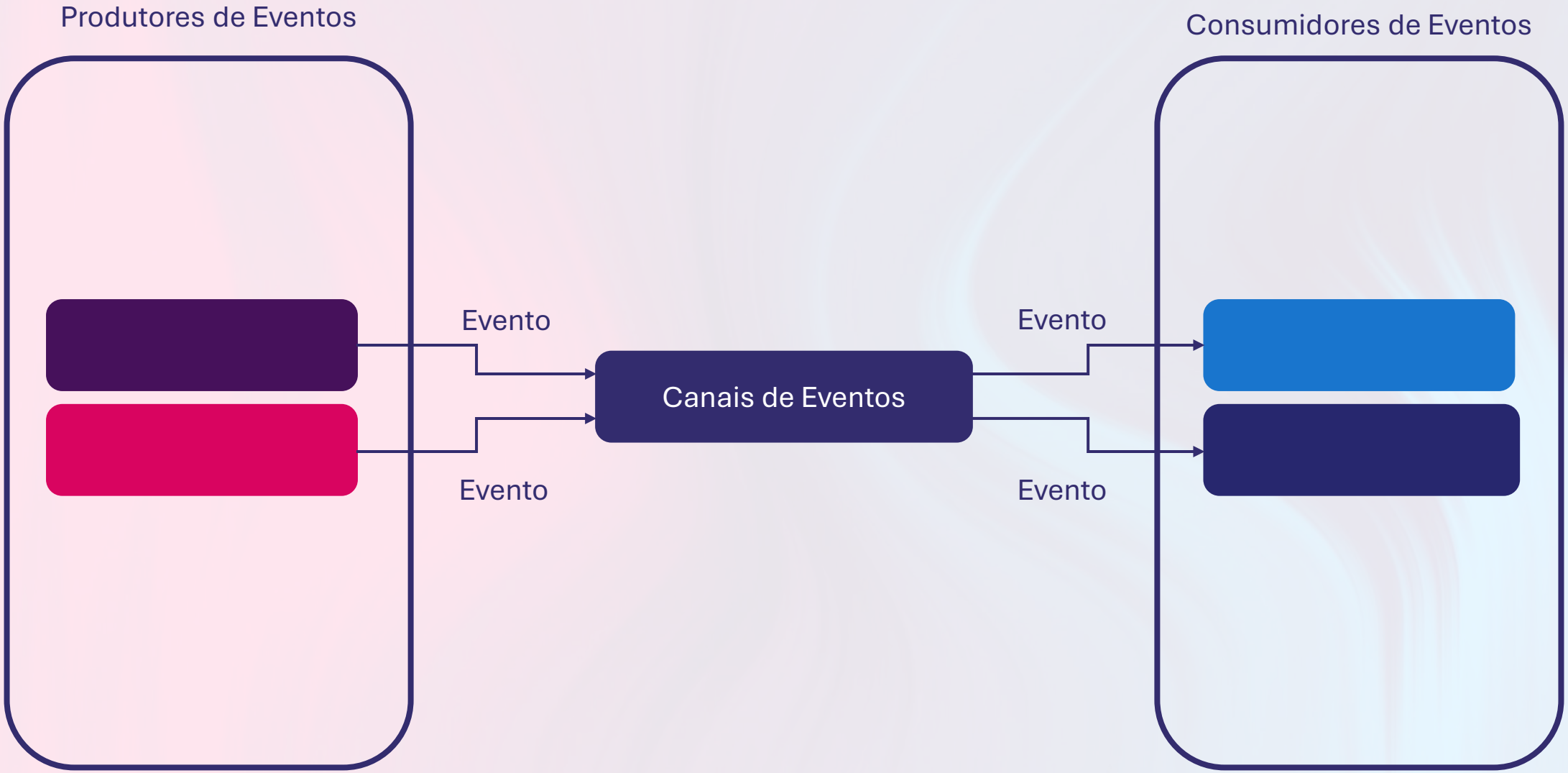
- Criar funções demasiado acopladas
- Espalhar lógica de negócio por muitos handlers
- Depender demasiado de serviços específicos do fornecedor
- Perder visão global do sistema

ARQUITETURA ORIENTADA A EVENTOS

EVENT – DRIVEN

VITOR MANUEL SANTOS

ARQUITETURA ORIENTADA A EVENTOS



ARQUITETURA ORIENTADA A EVENTOS

- Numa arquitetura event-driven, partes do sistema comunicam através de eventos.
- Um evento representa algo que já aconteceu e que pode interessar a outros componentes.

- Exemplos de eventos:
  - pedido criado
  - pagamento confirmado
  - encomenda expedida
  - erro de integração detetado
  - ação registada para auditoria

ARQUITETURA ORIENTADA A EVENTOS

- Em vez de um componente chamar diretamente outro, ele publica um evento.
- Outros componentes podem reagir a esse evento, se fizer sentido.

ARQUITETURA ORIENTADA A EVENTOS – VANTAGENS

- Reduz acoplamento direto
- Facilita reação assíncrona
- Permite adicionar novos consumidores sem alterar a origem
- Útil para integrações, notificações e auditoria

ARQUITETURA ORIENTADA A EVENTOS – DESVANTAGENS

- Maior dificuldade de rastreamento
- Consistência mais difícil de gerir
- Debugging mais complexo
- Exige boa observabilidade e desenho de eventos

ARQUITETURA ORIENTADA A EVENTOS

- Em arquitetura orientada a eventos, a organização muda bastante face ao modelo clássico **controller → service → repository → BD**
- A ideia principal é a seguinte:
  - o controller recebe um pedido
  - em vez de coordenar tudo de forma direta e síncrona, dispara uma ação
  - essa ação pode gravar algo e depois publicar um evento
  - outros módulos/serviços ouvem esse evento e reagem

ARQUITETURA ORIENTADA A EVENTOS

- Fluxo exemplar:

  Controller → Application Service / Command Handler → BD local → Evento publicado

  PaymentController

  → CreatePaymentService

  → grava Payment na BD de Pagamentos

  → publica PaymentCreated

ORIENTADA A EVENTOS – EXEMPLO DE IMPLEMENTAÇÃO

Lado de escrita

- Responsável por:
  - receber comandos
  - aplicar regras de negócio
  - alterar estado
  - publicar eventos

- Exemplos:
  - CreatePaymentService
  - ConfirmPaymentService
  - CancelOrderService

- Estes services tratam de coisas como:
  - pode criar?
  - pode mudar de estado?
  - que evento deve sair?

ORIENTADA A EVENTOS – EXEMPLO DE IMPLEMENTAÇÃO

Lado de consumo de eventos

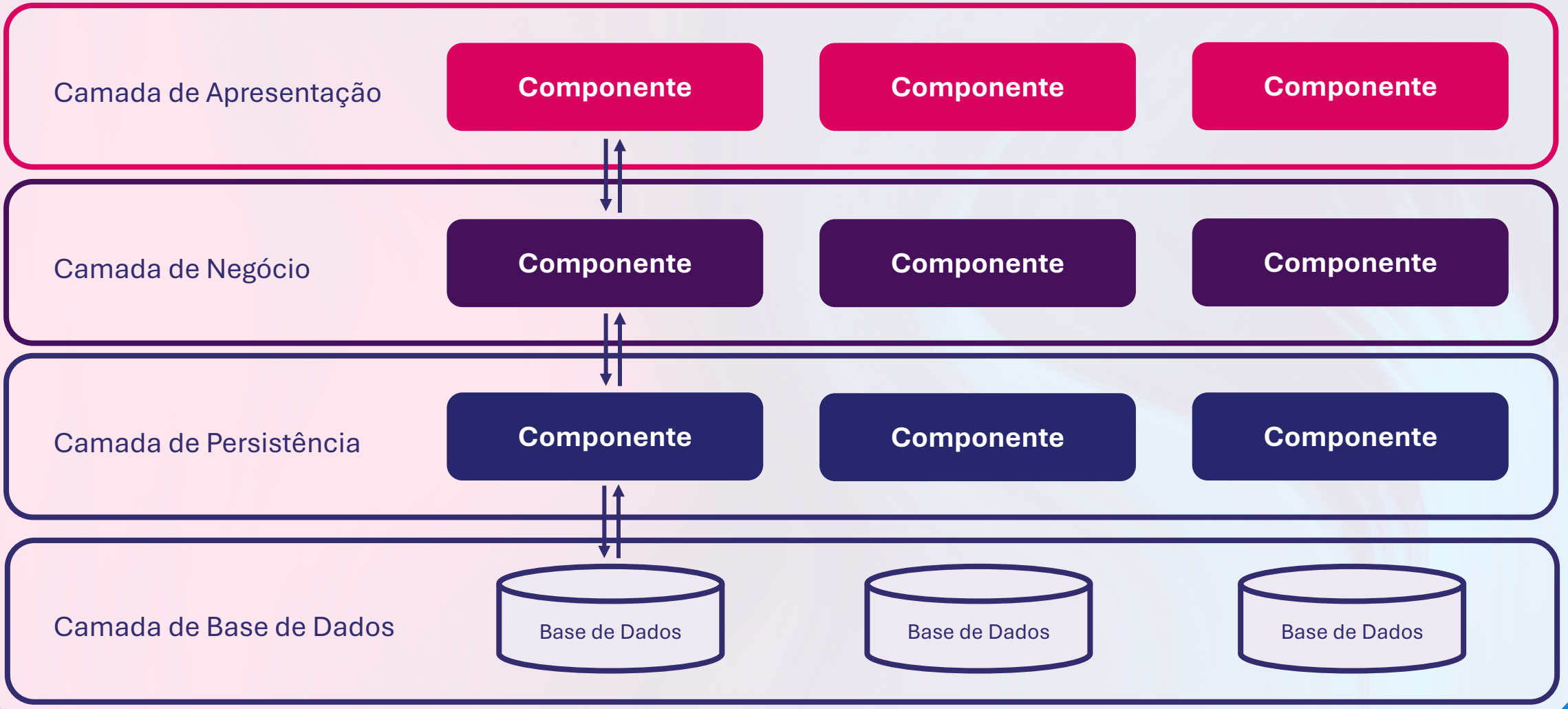
- Responsável por:
  - ouvir eventos do broker
  - reagir a esses eventos
  - atualizar dados locais
  - lançar novos eventos, se necessário

- Exemplos:
  - CustomerCreatedConsumer
  - PaymentConfirmedConsumer
  - OrderScheduledConsumer

ARQUITETURA EM CAMADAS

VITOR MANUEL SANTOS

ARQUITETURA EM CAMADAS



ARQUITETURA EM CAMADAS

- **Camadas típicas**
  - Apresentação — interface, API, controllers
  - Aplicação/Negócio — coordenação de casos de uso
  - Domínio — regras de negócio
  - Infraestrutura/Base de Dados — base de dados, email, APIs externas, filas

ARQUITETURA EM CAMADAS – VANTAGENS

- Estrutura fácil de compreender
- Boa base para equipas em crescimento
- Promove organização e separação de preocupações
- Adequada para muitos sistemas empresariais

ARQUITETURA EM CAMADAS – RISCOS

- Pode gerar excesso de dependência vertical
- Pode esconder regras de negócio em camadas erradas
- Às vezes favorece organização técnica em vez de organização por domínio

ARQUITETURA HEXAGONAL

PORTS & ADAPTERS

VITOR MANUEL SANTOS

ARQUITETURA HEXAGONAL

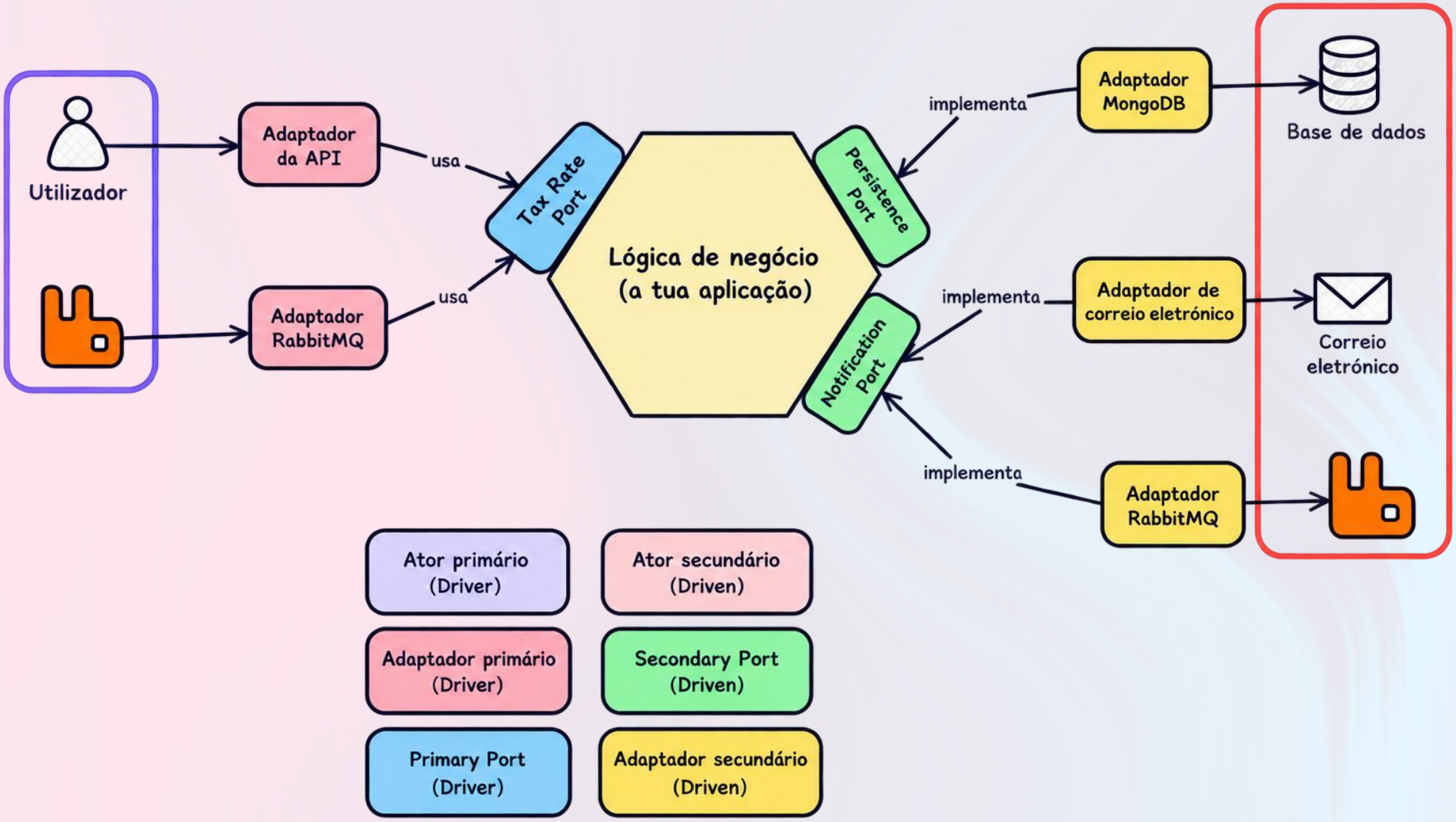
- A arquitetura hexagonal (também conhecida como Ports & Adapters) organiza o sistema para que o núcleo de negócio fique isolado das tecnologias externas. Permite que uma aplicação seja igualmente acionada por utilizadores, programas, testes automatizados ou scripts batch, e que possa ser desenvolvida e testada de forma isolada relativamente aos seus dispositivos de execução e bases de dados finais.
- Foi concebida por Alistair Cockburn em 1994, mas escrita oficialmente no blog da sua autoria em 2005.

- **Conceitos**
  - **Port** = contrato/interface de comunicação
  - **Adapter** = implementação concreta que liga o sistema ao exterior

ARQUITETURA HEXAGONAL

- Permite que a lógica de negócio não dependa diretamente de:
  - base de dados
  - Frameworks
  - APIs externas
  - interfaces web
  - sistemas de mensagens

ARQUITETURA HEXAGONAL



ARQUITETURA HEXAGONAL – O HEXÁGONO

- Antes de se entrarem em detalhes sobre este conceito, importa compreender por que razão o hexágono foi a forma escolhida. A explicação reside no facto de, durante muito tempo, os diagramas de arquitetura utilizarem sobretudo quadrados, tendo Alistair procurado uma forma simétrica que ainda não estivesse associada a um padrão visual dominante e que permitisse representar portas e adaptadores com maior liberdade.
- No que respeita à finalidade do hexágono, é no seu interior que se encontra a lógica de negócio central, incluindo os casos de uso e, eventualmente, construções do domínio, como entidades de domínio, entre outras.

ARQUITETURA HEXAGONAL – O HEXÁGONO

- Observando o diagrama de arquitetura, verifica-se que tudo o que se encontra do lado esquerdo é designado por Driver X, sendo X o elemento em causa (por exemplo, driver adapter). Esta designação resulta do facto de esse lado estar relacionado com a utilização da aplicação.
- Por sua vez, o lado direito é designado por Driven X, uma vez que corresponde aos elementos que a própria aplicação utiliza.
- Pode então colocar-se a seguinte questão: como poderá esta lógica de negócio ser invocada a partir do exterior? E como poderá a aplicação recorrer a bases de dados ou a message brokers? É precisamente neste ponto que surge o conceito de **portas**.

ARQUITETURA HEXAGONAL – PORTAS

- As portas são interfaces definidas pelo hexágono. Os adaptadores externos ao hexágono implementam essas interfaces, no caso das portas driven, ou consomem-nas, no caso das portas driving.
- Existem dois tipos de portas: as portas driver e as portas driven.

ARQUITETURA HEXAGONAL – PORTAS

- No caso das portas driver, estas são utilizadas para invocar os casos de uso. Por exemplo, podem ser usadas para chamar um caso de uso responsável por debitar dinheiro da conta bancária de um cliente.
- As portas driven, por sua vez, são utilizadas pela aplicação para invocar os atores driven. Por exemplo, uma destas portas pode representar o contrato responsável por armazenar, recuperar, atualizar e eliminar objetos numa base de dados.
- As portas podem ainda ser organizadas de acordo com a sua intenção. Por exemplo, poderá existir um grupo de portas destinado a notificações e outro orientado para tarefas administrativas.

ARQUITETURA HEXAGONAL – PORTAS

- Um equívoco frequente consiste em assumir que, por se tratar de um hexágono, é obrigatório existirem exatamente seis portas. Tal interpretação não é correta. O número de portas não está limitado pela forma geométrica da representação, podendo existir tantas quantas forem necessárias.
- Definidos os contratos para interagir com a aplicação e para aceder ao exterior, surge naturalmente outra questão: quem utiliza esses contratos? É neste contexto que entram os **adaptadores**.

ARQUITETURA HEXAGONAL – ADAPTADORES

- Os adaptadores são os elementos intermédios entre os atores e as portas. Os adaptadores driver devem estar acoplados a um ator específico. Por exemplo, um adaptador driver deve estar associado a **atores driver**, de modo a converter os dados enviados por esses atores em dados compatíveis com as entradas dos casos de uso.
- Por outro lado, os adaptadores driven devem converter os dados recebidos da aplicação para um formato compreensível pelos atores driven, realizando igualmente a conversão no sentido inverso sempre que necessário.

ARQUITETURA HEXAGONAL – ATORES DRIVER

- São os elementos que pretendem utilizar a aplicação através dos seus casos de uso. Como exemplo, podem referir-se um utilizador num portal Web a executar tarefas administrativas ou até um message broker que procura introduzir mensagens na aplicação.

ARQUITETURA HEXAGONAL – ATORES DRIVEN

- Correspondem aos elementos utilizados pelos casos de uso, como bases de dados ou message brokers.

ARQUITETURA HEXAGONAL – VANTAGENS

- facilita testes
- protege o domínio
- melhora desacoplamento técnico
- torna substituições tecnológicas menos dolorosas

ARQUITETURA HEXAGONAL – RISCOS

- pode parecer mais abstrata no início
- exige disciplina de desenho
- pode ser exagerada em sistemas muito simples

DECOMPOSIÇÃO POR DOMÍNIO VS CAMADA TÉCNICA

VITOR MANUEL SANTOS

DECOMPOSIÇÃO POR DOMÍNIO VS CAMADA TÉCNICA

Antes de dividir o sistema, importa saber: dividir segundo quê?

- Quando um sistema cresce, é natural tentar organizá-lo melhor.
- Mas há duas lógicas muito diferentes de decomposição:
  - por camada técnica
  - por domínio de negócio

DECOMPOSIÇÃO POR DOMÍNIO VS CAMADA TÉCNICA

- A forma como o sistema é dividido influencia:
  - Acoplamento
  - Manutenção
  - Evolução
  - Autonomia das equipas
  - Clareza das responsabilidades

DECOMPOSIÇÃO POR CAMADA TÉCNICA

- Na decomposição por camada técnica, o sistema é organizado segundo tipos de componente, por exemplo:
  - Controllers
  - Services
  - Repositories
  - Entities
  - views

DECOMPOSIÇÃO POR DOMÍNIO

- Na decomposição por domínio, o sistema é organizado segundo áreas de negócio ou capacidades funcionais, por exemplo:
  - Pedidos
  - Clientes
  - Equipamentos
  - Faturação
  - Notificações
  - Auditoria
- Cada área pode conter os seus próprios:
  - Controllers
  - Services
  - Repositories
  - Modelos

DECOMPOSIÇÃO POR DOMÍNIO VS CAMADA TÉCNICA

Por camada técnica

/controllers
- PedidoController
- ClienteController
- FaturacaoController

/services
- PedidoService
- ClienteService
- FaturacaoService

/repositories
- PedidoRepository
- ClienteRepository
- FaturacaoRepository

Por domínio

/pedidos
- PedidoController
- PedidoService
- PedidoRepository
- Pedido

/clientes
- ClienteController
- ClienteService
- ClienteRepository
- Cliente

/faturacao
- FaturacaoController
- FaturacaoService
- FaturacaoRepository
- Fatura

DECOMPOSIÇÃO POR CAMADA TÉCNICA – VANTAGENS

- Vantagens:
  - fácil de compreender no início
  - familiar para equipas iniciantes
  - simples de montar rapidamente
  - útil em sistemas pequenos ou protótipos
  - promove separação técnica básica
- Pode funcionar bem quando:
  - o sistema ainda é reduzido
  - o domínio é simples
  - a equipa está a começar
  - a prioridade é arrancar rapidamente

DECOMPOSIÇÃO POR CAMADA TÉCNICA – LIMITAÇÕES

- o domínio de negócio fica espalhado por várias pastas
- torna-se mais difícil perceber quem “é dono” de cada capacidade
- alterações numa funcionalidade obrigam a saltar entre muitas zonas do projeto
- favorece acoplamento indireto entre áreas do sistema
- pode esconder fronteiras de negócio importantes

DECOMPOSIÇÃO POR DOMÍNIO – VANTAGENS

- Vantagens:
  - responsabilidades mais claras
  - melhor alinhamento com o negócio
  - fronteiras mais visíveis
  - manutenção mais focada
  - base mais forte para modularidade
  - facilita evolução para equipas mais autónomas
- Benefício prático:
  - Endpoints
  - Lógica
  - Acesso a dados
  - Regras específicas
  - Objetos relevantes

BOUNDARIES, RESPONSABILIDADES E ACOPLAMENTO

VITOR MANUEL SANTOS

BOUNDARIES, RESPONSABILIDADES E ACOPLAMENTO

- Como perceber se a divisão do sistema está saudável.
  - **Boundaries**
  - **Responsabilidades**
  - **Acoplamento**

BOUNDARIES

- Um boundary representa um limite claro entre partes do sistema.
- Esse limite ajuda a definir:
  - quem é responsável por quê
  - que regras pertencem a cada área
  - que dados cada parte deve controlar
  - como a comunicação entre áreas deve acontecer
- Exemplos de boundaries: Pedidos, Clientes, Faturação, Notificações, Auditoria

RESPONSABILIDADES BEM DEFINIDAS

- Uma parte do sistema deve ter uma responsabilidade compreensível e relativamente focada.
- Sinais de boa atribuição de responsabilidades:
  - é fácil explicar o papel do módulo ou serviço
  - as regras dessa área vivem maioritariamente no mesmo sítio
  - as mudanças nessa capacidade têm impacto mais localizado
  - há menos ambiguidade sobre ownership
- Exemplo: O módulo Pedidos deve ser responsável por: criar pedido, validar pedido, mudar estado do pedido, registar histórico do pedido

ACOPLAMENTO

- Há acoplamento quando uma parte do sistema depende fortemente de outra para funcionar, evoluir ou ser alterada.
- Exemplos:
  - um módulo conhece demasiados detalhes internos de outro
  - uma alteração numa funcionalidade obriga a mexer em várias áreas
  - serviços dependem constantemente uns dos outros
  - várias partes partilham diretamente os mesmos dados sem controlo claro
- Algum acoplamento é inevitável. O problema é o acoplamento excessivo ou desnecessário.

SINAIS DE BOUNDARIES FRACOS

- Regras de negócio espalhadas por vários módulos
- Múltiplas áreas alteram os mesmos dados sem coordenação clara
- Dúvidas constantes sobre “onde deve ficar isto?”
- Dependências cruzadas frequentes
- Funcionalidades que exigem mudanças em muitos pontos ao mesmo tempo

SINAIS DE BOUNDARIES FRACOS

- Criar um pedido implica mexer diretamente em:
  - Pedidos
  - Stock
  - Notificações
  - Faturação
  - Auditoria
- Tudo no mesmo fluxo e sem fronteiras claras.

BOUNDARIES SAUDÁVEIS VS BOUNDARIES FRACOS

- Boundaries mais saudáveis:
  - responsabilidades claras
  - dependências mais controladas
  - comunicação explícita
  - ownership dos dados mais visível
  - impacto das mudanças mais contido
- Boundaries mais frágeis:
  - responsabilidades ambíguas
  - dependências escondidas
  - acesso direto a dados “alheios”
  - lógica espalhada
  - regressões frequentes

RELAÇÕES COM OS ESTILOS ARQUITETURAIS

Monólito tradicional

- Pode sofrer mais facilmente com boundaries difusos.

Monólito modular

- Tenta reforçar boundaries dentro da mesma aplicação.

Microsserviços

- Exigem boundaries ainda mais claros, porque os custos de uma má divisão são maiores.

Event-driven

- Pode reduzir acoplamento direto, mas não corrige boundaries mal definidos por si só.

Nenhum estilo resolve automaticamente mau desenho de responsabilidades

OWNERSHIP DOS DADOS

- Uma forma muito útil de avaliar boundaries é perguntar:

Quem deve ser dono deste dado?

- Exemplos:
  - dados de pedido → módulo/serviço de Pedidos
  - dados de faturação → módulo/serviço de Faturação
  - dados de notificação → módulo/serviço de Notificações

- Um problema frequente é quando várias áreas escrevem livremente sobre os mesmos dados:
  - aumenta o acoplamento
  - cresce a confusão
  - torna-se mais difícil garantir consistência

CONTINUANDO A INTEGRAR IA NESTE PROCESSO...

VITOR MANUEL SANTOS

AVALIAR ARQUITETURAS COM IA

A primeira proposta não deve ser a decisão final. Workflow recomendado:



ARCHITECTURE DECISION RECORDS (ADR)

Documentar o porquê, não apenas o quê.

- Um ADR regista uma decisão arquitetural e as razões que a justificam.
- Estrutura de um ADR:
  - Título: Decisão arquitetural.
  - Contexto: Problema e restrições existentes.
  - Alternativas: Soluções consideradas.
  - Decisão: Solução escolhida e justificação.
  - Consequências: Benefícios, riscos e compromissos.
  - Estado: Proposed / Accepted / Deprecated / Superseded.

ARCHITECTURE DECISION RECORDS (ADR)

- Exemplo de organização:

```
docs/
└── adr/
    ├── ADR-001-architecture.md
    ├── ADR-002-database.md
    └── ADR-003-authentication.md
```

- Com IA: Gerar alternativas, comparar consequências e estruturar a documentação da decisão.
- Princípio: A IA pode ajudar a justificar uma decisão, mas não deve inventar os seus fundamentos.

(Para mais profundidade sobre ADRs, como criar, organizar e gerir, consultar material de apoio: **05 - ADRs**)

MODELAÇÃO DE DADOS

VITOR MANUEL SANTOS

MODELAÇÃO DE DADOS ASSISTIDA POR IA

- A IA pode apoiar:
  - Identificação de **entidades** e **atributos**.
  - Definição de **relacionamentos** e **cardinalidades**.
  - Identificação de **chaves** primárias e estrangeiras.
  - Aplicação de regras de integridade.
  - **Normalização** e otimização do modelo.
  - Geração de diagramas ER e scripts SQL.
- Workflow recomendado:

  Requirements → Entities → Relationships → Constraints → ER Diagram → SQL
- Atenção que um modelo de dados tecnicamente válido não é necessariamente adequado às regras de negócio.

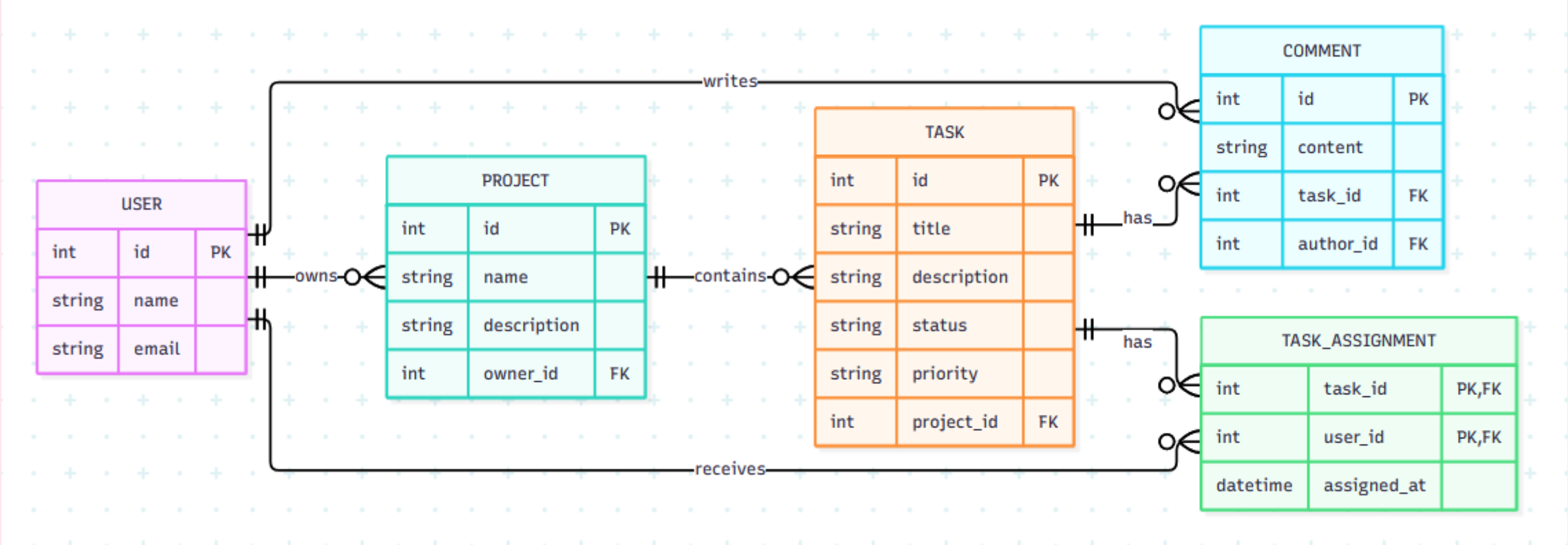
IDENTIFICAÇÃO DE ENTIDADES E RELACIONAMENTOS

- Exemplo: **Matriz** — Gestão de Projetos e Tarefas

Entidade	Principais atributos
User	id, name, email
Project	id, name, description, owner_id
Task	id, title, status, priority, project_id
Comment	id, content, task_id, author_id
TaskAssignment	task_id, user_id, assigned_at

- Relacionamentos:
  - User 1:N Project
  - Project 1:N Task
  - Task 1:N Comment
  - User N:M Task (através de TaskAssignment)

ENTITY-RELATIONSHIP DIAGRAM (ERD)



PROMPT & CONTEXT ENGINEERING P/ MODELAÇÃO DE DADOS

- Antes de pedir à IA para criar o modelo, devemos disponibilizar os artefactos produzidos anteriormente, como:

Contexto	Origem
Requisitos funcionais	`docs/requirements.md`
User Stories	`docs/user-stories.md`
Restrições	`docs/constraints.md`
Instruções do projeto	`AGENTS.md`
Decisões arquiteturais	`docs/architecture.md` e ADRs, quando disponíveis

- Também é importante conhecer os tipos de utilizador, as permissões, as regras de atribuição de tarefas e os requisitos de histórico ou auditoria, caso estejam definidos.
- Exemplo de ambiguidade: Uma tarefa pode ser atribuída a um único utilizador ou a vários? A resposta altera diretamente a cardinalidade do relacionamento e pode justificar uma entidade TaskAssignment.

PROMPT & CONTEXT ENGINEERING P/ MODELAÇÃO DE DADOS

- Pode consultar os exemplos de prompts usados e resultados, na pasta **06 - Anexos**

MERMAID – DIAGRAMAS COMO CÓDIGO

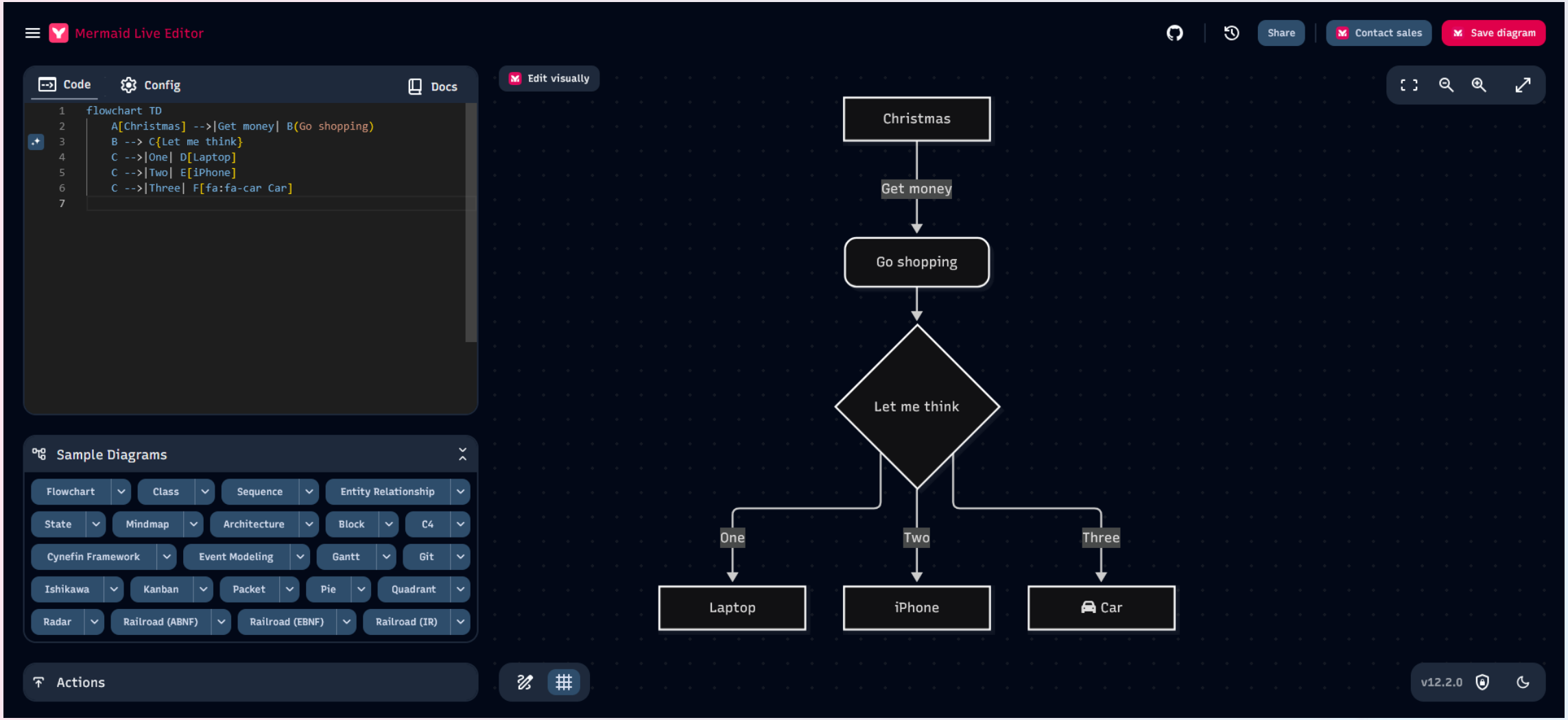
- O **Mermaid** é uma ferramenta open-source que permite criar diagramas através de uma sintaxe textual simples, sem necessidade de os desenhar manualmente.

- Como funciona?
  - Os diagramas são definidos através de código declarativo.
  - O Mermaid interpreta o código e gera automaticamente uma representação visual.
  - Suporta diagramas ER, fluxogramas, diagramas de sequência, diagramas de classes, entre outros.
  - Permite alterar e atualizar diagramas simplesmente editando o código.
  - Pode ser integrado em documentação Markdown, GitHub e outras ferramentas.

MERMAID – DIAGRAMAS COMO CÓDIGO

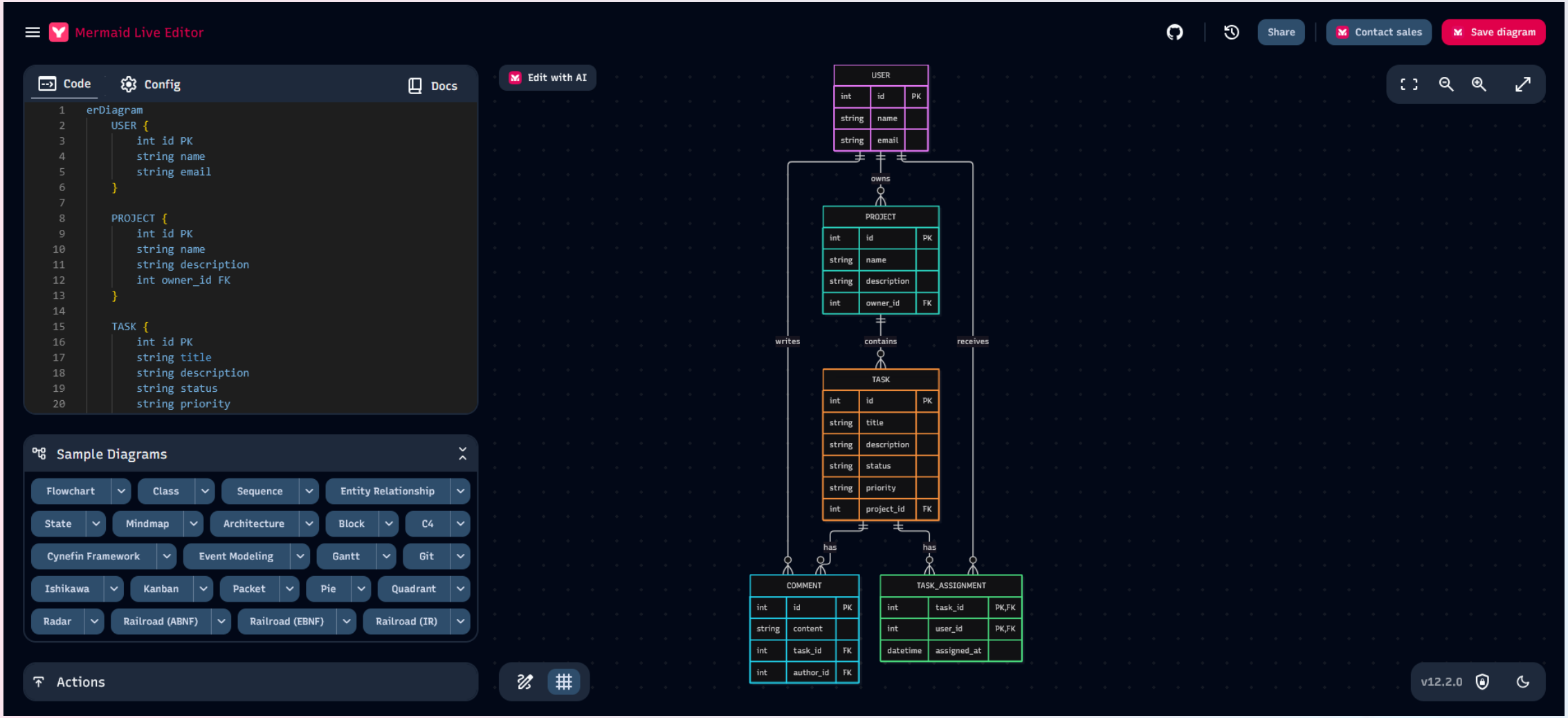
- Vantagens na Engenharia com IA:
  - A IA pode gerar diagramas diretamente a partir de requisitos em linguagem natural.
  - Facilita a documentação e a manutenção da arquitetura.
  - Permite versionar diagramas juntamente com o código-fonte.
- **Ferramentas online**: Mermaid Live Editor & Mermaid AI
  - https://mermaid.live
  - https://mermaid.ai

MERMAID LIVE



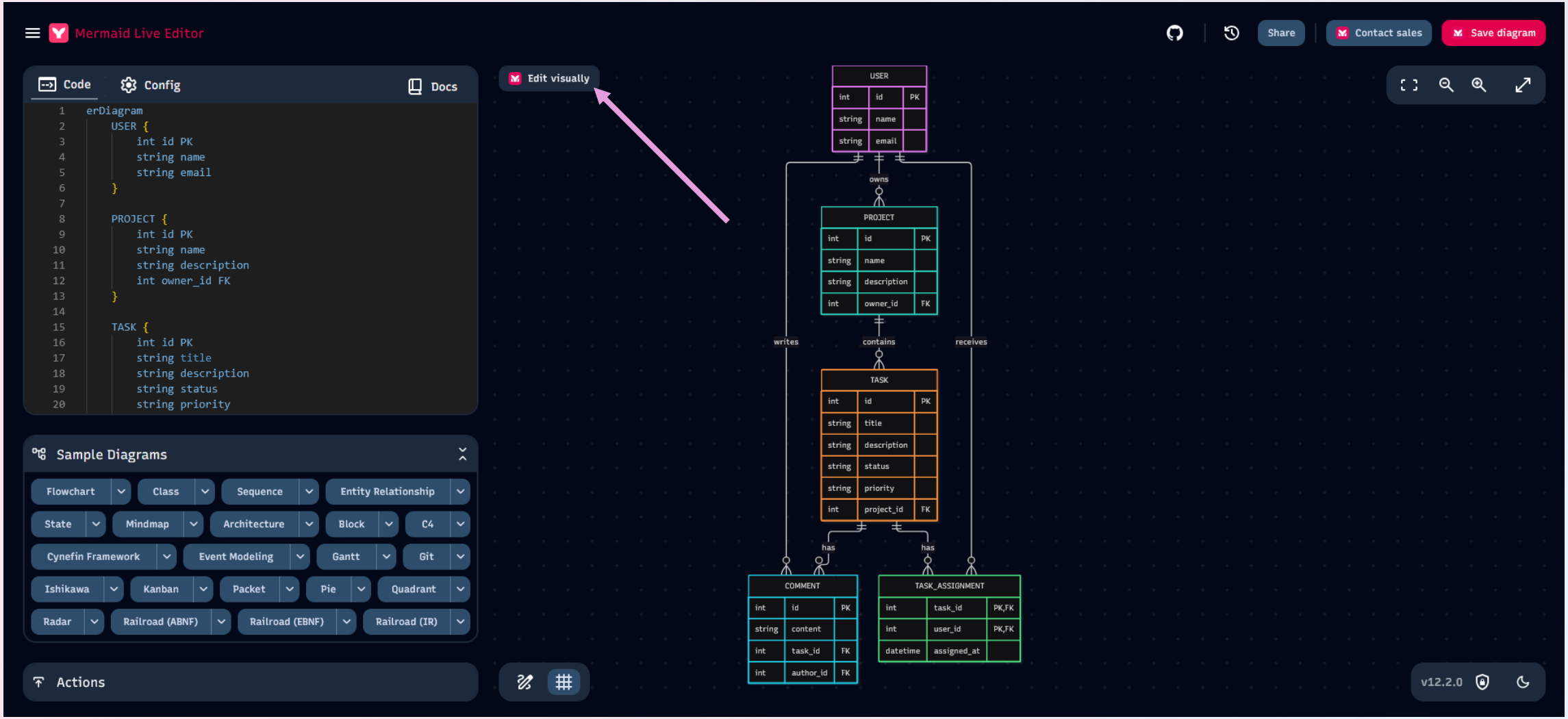
VITOR MANUEL SANTOS

MERMAID LIVE



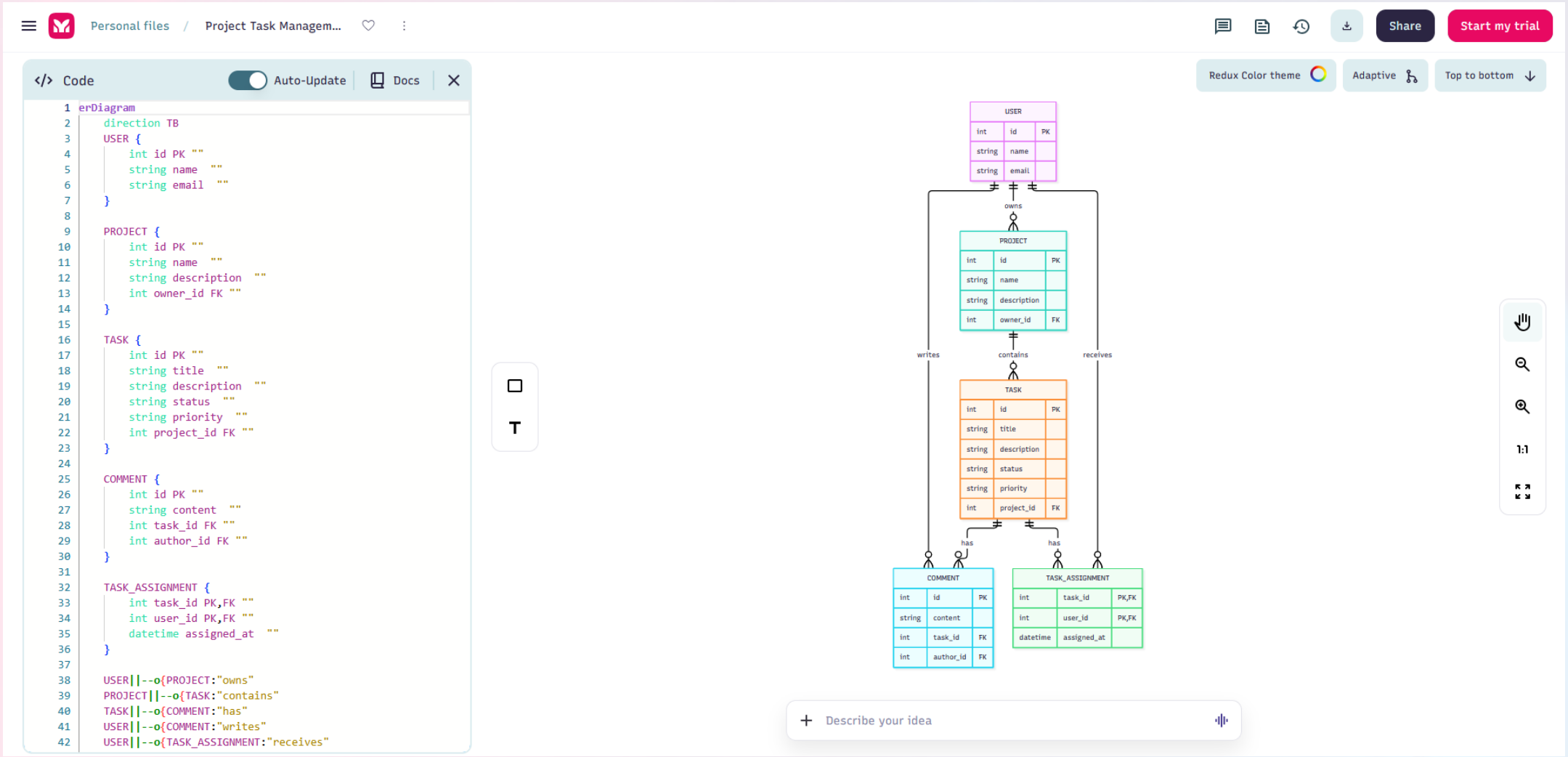
VITOR MANUEL SANTOS

MERMAID LIVE



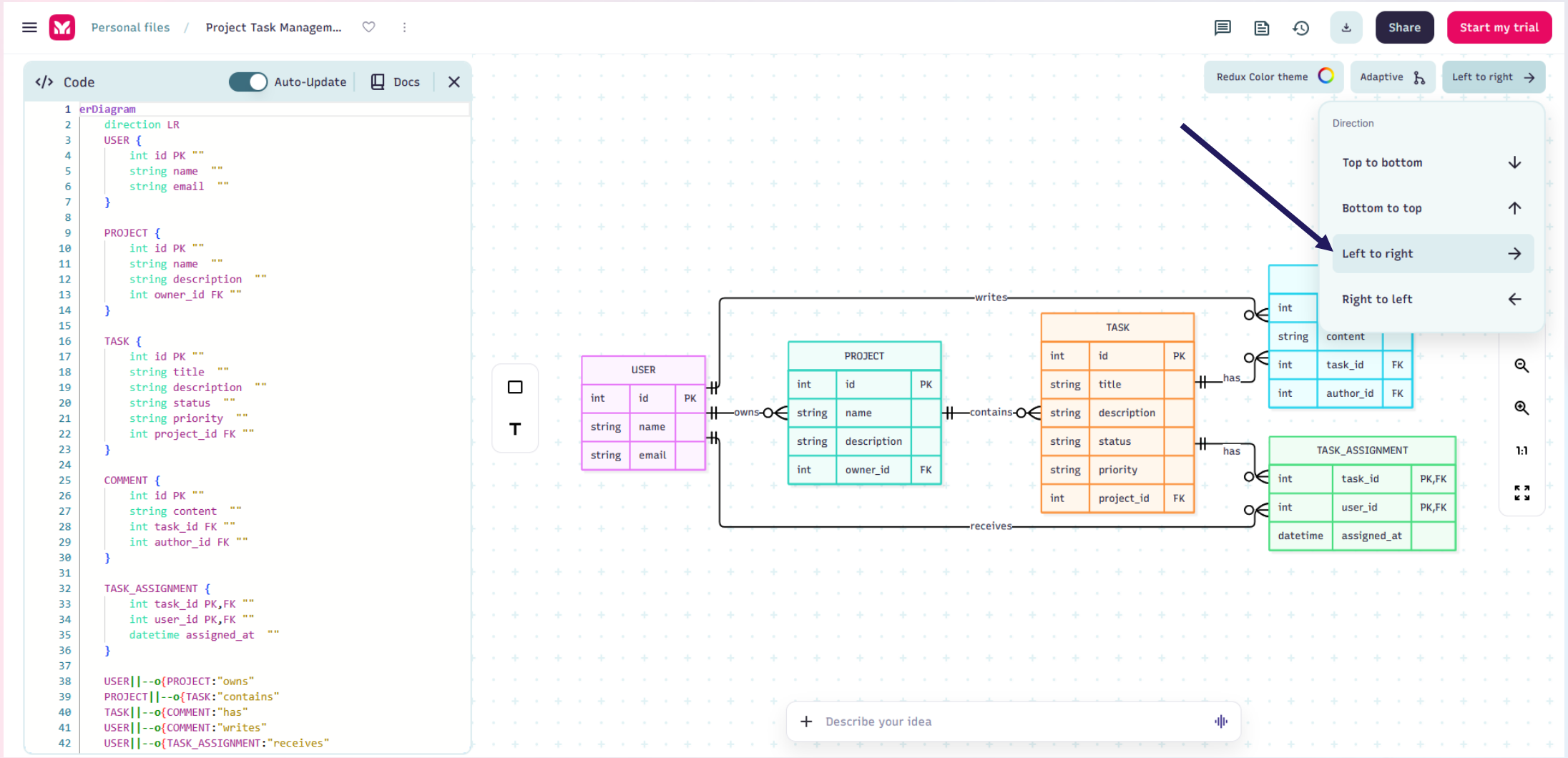
VITOR MANUEL SANTOS

MERMAID AI



VITOR MANUEL SANTOS

MERMAID AI




```
erDiagram
    direction LR
    USER {
        int id PK ""
        string name  ""
        string email  ""
    }

    PROJECT {
        int id PK ""
        string name  ""
        string description  ""
        int owner_id FK ""
    }

    TASK {
        int id PK ""
        string title  ""
        string description  ""
        string status  ""
        string priority  ""
        int project_id FK ""
    }

    COMMENT {
        int id PK ""
        string content  ""
        int task_id FK ""
        int author_id FK ""
    }

    TASK_ASSIGNMENT {
        int task_id PK,FK ""
        int user_id PK,FK ""
        datetime assigned_at  ""
    }

    USER||--o{PROJECT:"owns"
    PROJECT||--o{TASK:"contains"
    TASK||--o{COMMENT:"has"
    USER||--o{COMMENT:"writes"
    USER||--o{TASK_ASSIGNMENT:"receives"
```

VITOR MANUEL SANTOS

DO MODELO DE DADOS AO SQL

Transformar o modelo conceptual em estruturas persistentes.

- Dependendo da arquitetura podemos optar por persistência de dados relacional ou não-relacional.
- Caso optemos por uma arquitetura mais tradicional que inclua um paradigma relacional, iremos, com certeza criar uma Base de Dados usando **SQL**.

DO MODELO DE DADOS AO SQL

- Excerto de exemplo de código de criação de algumas entidades da Matriz:

```
CREATE TABLE projects (
    id BIGINT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    description TEXT,
    owner_id BIGINT NOT NULL,
    FOREIGN KEY (owner_id)
    REFERENCES users(id)
);
```

```
CREATE TABLE tasks (
    id BIGINT PRIMARY KEY,
    project_id BIGINT NOT NULL,
    title VARCHAR(200) NOT NULL,
    status VARCHAR(30) NOT NULL,
    priority VARCHAR(20),
    FOREIGN KEY (project_id)
    REFERENCES projects(id)
);
```

DO MODELO DE DADOS AO SQL

- Aspetos importantes de validar:
  - Data types e nullability.
  - Primary Keys e Foreign Keys.
  - Constraints (UNIQUE, CHECK, NOT NULL).
  - Indexes e desempenho.
  - Referential integrity.
  - Migrations e versionamento do schema.
- Com IA: Gerar SQL, migrations, dados de teste (seed data) e propor índices com base nos padrões de consulta.

VALIDAÇÃO E OTIMIZAÇÃO DO MODELO

Dimensão	Exemplos de questões de validação
Integridade	Existem relações ou constraints em falta?
Normalização	Há redundância ou anomalias de atualização?
Performance	Os índices suportam as queries mais frequentes?
Escalabilidade	Como evolui o modelo com o volume de dados?
Segurança	Existem dados sensíveis? Como são protegidos?
Manutenção	O schema suporta alterações futuras?

VALIDAÇÃO E OTIMIZAÇÃO DO MODELO

- Exemplo de prompt:
  - Analisa este modelo de dados como um Database Architect. Identifica problemas de normalização, integridade referencial, performance e segurança. Considera operações frequentes de consulta por projeto, estado e responsável. Propõe melhorias, justifica cada alteração e identifica os respetivos trade-offs.
- A IA pode identificar oportunidades de otimização, mas a validação deve considerar queries reais, volumes de dados e regras de negócio.

API

VITOR MANUEL SANTOS

O QUE É UMA API?

- **API** = Application Programming Interface
- Conjunto de **regras** e **métodos** para um programa falar com outro
- Define como pedir dados e como os dados são devolvidos
- Evita acesso direto à base de dados / lógica interna

ANALOGIA

- **Cliente**: aplicação cliente (frontend, app, etc.)
- **Cozinha**: servidor / backend
- **Empregado de mesa**: API

- O cliente nunca fala diretamente com o cozinheiro
- Faz o **pedido** ao empregado → ele leva à cozinha → traz a comida (**resposta**)

PROPÓSITOS DE UMA API

- Permitem que **sistemas diferentes comuniquem**
- **Reutilização**: vários clientes podem usar a mesma API
- **Segurança**: expõe só o que é necessário
- **Escalabilidade**: apps web, mobile, integrações com outras empresas
- Ex.: APIs do Google como Google Maps, Facebook, MBWay, Paypal, OpenWeather, Kayak, etc.

APIS NA WEB (HTTP)

- Muitas APIs funcionam sobre o **protocolo HTTP**
- Cliente envia um HTTP **request**
- Servidor responde com um HTTP **response**
- Normalmente a API é acessível por URLs do tipo:

  https://api.exemplo.com/...

O QUE É UM ENDPOINT?

- **Endpoint** = ponta da API onde um cliente faz um pedido
- É um endereço específico (URL) que representa uma funcionalidade
- Exemplo:
  - /api/clientes: trabalhar com clientes
  - /api/produtos: trabalhar com produtos

ENDPOINT = URL + MÉTODO

- Um **endpoint** não é só a URL, é combinação de:
  - **URL** (ex.: /api/clientes)
  - **Método HTTP** (ex.: GET, POST, ...)
- Exemplos:
  - GET /api/clientes → listar clientes
  - POST /api/clientes → criar cliente
  - GET /api/clientes/10 → pesquisar cliente com ID 10
  - DELETE /api/clientes/10 → apagar cliente com ID 10

MÉTODOS HTTP PRINCIPAIS

- **GET** → Ler/obter dados
- **POST** → Criar um novo recurso
- **PUT** → Atualizar (substituir) um recurso existente
- **PATCH** → Atualizar parcialmente
- **DELETE** → Apagar um recurso

REQUEST - PEDIDO

- Um HTTP Request tem normalmente:
  - **URL** → para que endpoint estamos a falar
  - **Método HTTP** → que operação queremos fazer
  - **Headers** → informações extra (ex.: Content-Type, Authorization)
  - **Body** (corpo) → dados enviados (ex.: num POST com um novo cliente)
  - **Parâmetros** → filtros, IDs, pesquisa, etc.

EXEMPLO BODY - JSON

```
{
    "nome“ : "Ana Santos“ ,
    "email“ : "ana@example.com"
}
```

RESPONSE - RESPOSTA

- Um **HTTP Response** normalmente inclui:
  - **Status code** (código do resultado)
  - **Headers**
  - **Body** com os dados (normalmente JSON)

EXEMPLO RESPOSTA - JSON

```
{
    “id” : 10 ,
    "nome“ : "Ana Santos“ ,
    "email“ : "ana@example.com"
}
```

STATUS CODE - CÓDIGOS DE ESTADO

- **200 OK** → correu bem
- **201 Created** → recurso criado com sucesso
- **400 Bad Request** → pedido mal formado
- **401 Unauthorized** → não autenticado
- **403 Forbidden** → autenticado mas sem permissão
- **404 Not Found** → recurso não encontrado
- **500 Internal Server Error** → erro no servidor

VITOR MANUEL SANTOS

RESOURCES - RECURSOS

- Um **recurso** é uma **entidade** (tipo de coisa) do sistema:
  - Cliente, Produto, Encomenda, Aluno, Curso, etc.
- Normalmente:
  - **Coleção**: /api/clientes
  - **Elemento**: /api/clientes/10
- A API REST organiza-se em torno destes recursos

PARÂMETROS: PATH VS QUERY

- **Path parameters** (no caminho):
  - Fazem parte da URL
  - Ex.: /api/clientes/10 → o 10 do cliente é o ID
- **Query parameters** (depois de ?):
  - Usados para filtros e pesquisa
  - Ex.: /api/clientes?ativo=true
  - Ex.: /api/produtos?categoria=livros&ordem=preco_desc

FORMATO DOS DADOS: JSON

- **JSON** = JavaScript Object Notation
- Fácil de ler por humanos e máquinas
- Suportado em quase todas as linguagens

EXEMPLO - JSON

```
{
    “id” : 1 ,
    "nome“ : “Produto XPTO“ ,
    “preco“ : 9.99 ,
    “ativo“ : true
}
```

AUTENTICAÇÃO E AUTORIZAÇÃO

- Nem todas as APIs são públicas
- **Autenticação** → confirmar quem és
  - Ex.: login, tokens, JWT, API keys
- **Autorização** → o que podes fazer
  - Ex.: só admins podem apagar, outros só podem ler

API DESIGN

VITOR MANUEL SANTOS

API DESIGN ASSISTIDO POR IA

- A IA pode apoiar:
  - Identificação de recursos e endpoints.
  - Definição de operações e métodos HTTP.
  - Estruturação de requests e responses.
  - Definição de DTOs e validações.
  - Tratamento de erros e status codes.
  - Geração de documentação OpenAPI.
- Abordagem API-First:

  Requirements → Resources → Endpoints → Contracts → OpenAPI → Implementation
- Princípio: Definir e validar o contrato antes de implementar a API.

REST API DESIGN – EXEMPLOS DE ENDPOINTS

Método	Endpoint
GET	/projects
POST	/projects
GET	/projects/{id}
PATCH	/projects/{id}
DELETE	/projects/{id}
GET	/projects/{id}/tasks
POST	/projects/{id}/tasks
PATCH	/tasks/{id}

REST API DESIGN

- Boas práticas:
  - URLs orientadas a recursos, não a ações.
  - Utilização adequada dos métodos HTTP.
  - Naming consistente e previsível.
  - Paginação, filtros e ordenação.
  - Versionamento e compatibilidade.
  - Autenticação e autorização por recurso.
- Exemplo com IA: Gerar propostas de endpoints com base nas User Stories e no modelo de dados aprovado.

DTOs, VALIDAÇÃO E HTTP STATUS CODES

Request DTO

```
{
    "title": "Implementar login",
    "description": "Autenticação",
    "priority": "HIGH“
}
```

Response DTO

```
{
    "id": 42,
    "title": "Implementar login",
    "status": "TODO",
    "priority": "HIGH“
}
```

DTOs, VALIDAÇÃO E HTTP STATUS CODES

Código	Significado
`200 OK`	Operação concluída
`201 Created`	Recurso criado
`204 No Content`	Sucesso sem conteúdo
`400 Bad Request`	Pedido inválido
`401 Unauthorized`	Autenticação necessária ou inválida
`403 Forbidden`	Permissão insuficiente
`404 Not Found`	Recurso não encontrado
`409 Conflict`	Conflito com o estado atual
`500 Internal Server Error`	Erro interno

OpenAPI: DOCUMENTAÇÃO E CONTRATOS

- A **OpenAPI** Specification (OAS) é um padrão aberto e independente de linguagem para descrever APIs HTTP e REST em um formato legível tanto por humanos quanto por máquinas.
- Permite:
  - Documentar endpoints, parâmetros e respostas.
  - Definir schemas de dados.
  - Gerar documentação interativa.
  - Facilitar a integração Frontend/Backend.
  - Gerar clientes e apoiar testes automatizados.
  - Validar a conformidade da implementação com o contrato.

EXEMPLO — OpenAPI 3.1

- Ferramentas:
  - Swagger UI
  - Swagger Editor: https://editor.swagger.io/
  - Postman

Exemplo simplificado para apresentação. Um contrato completo deve especificar schemas de resposta, erros, autenticação e restantes restrições relevantes.

```yaml
openapi: 3.1.0

info:
  title: Matriz API
  version: 1.0.0

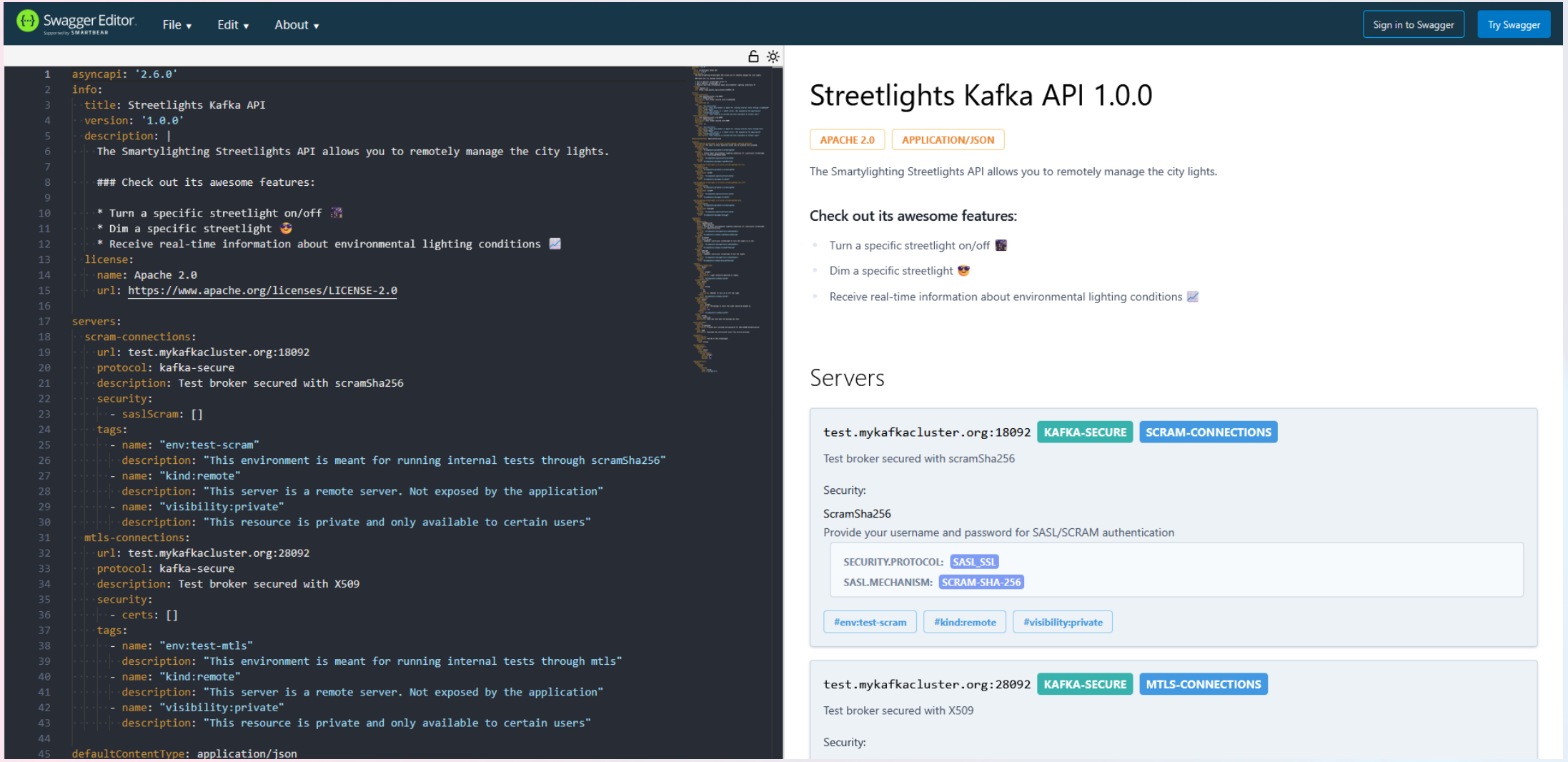
paths:
  /projects:
    get:
      summary: Listar projetos
      responses:
        '200':
          description: Lista de projetos

    post:
      summary: Criar projeto
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CreateProject'

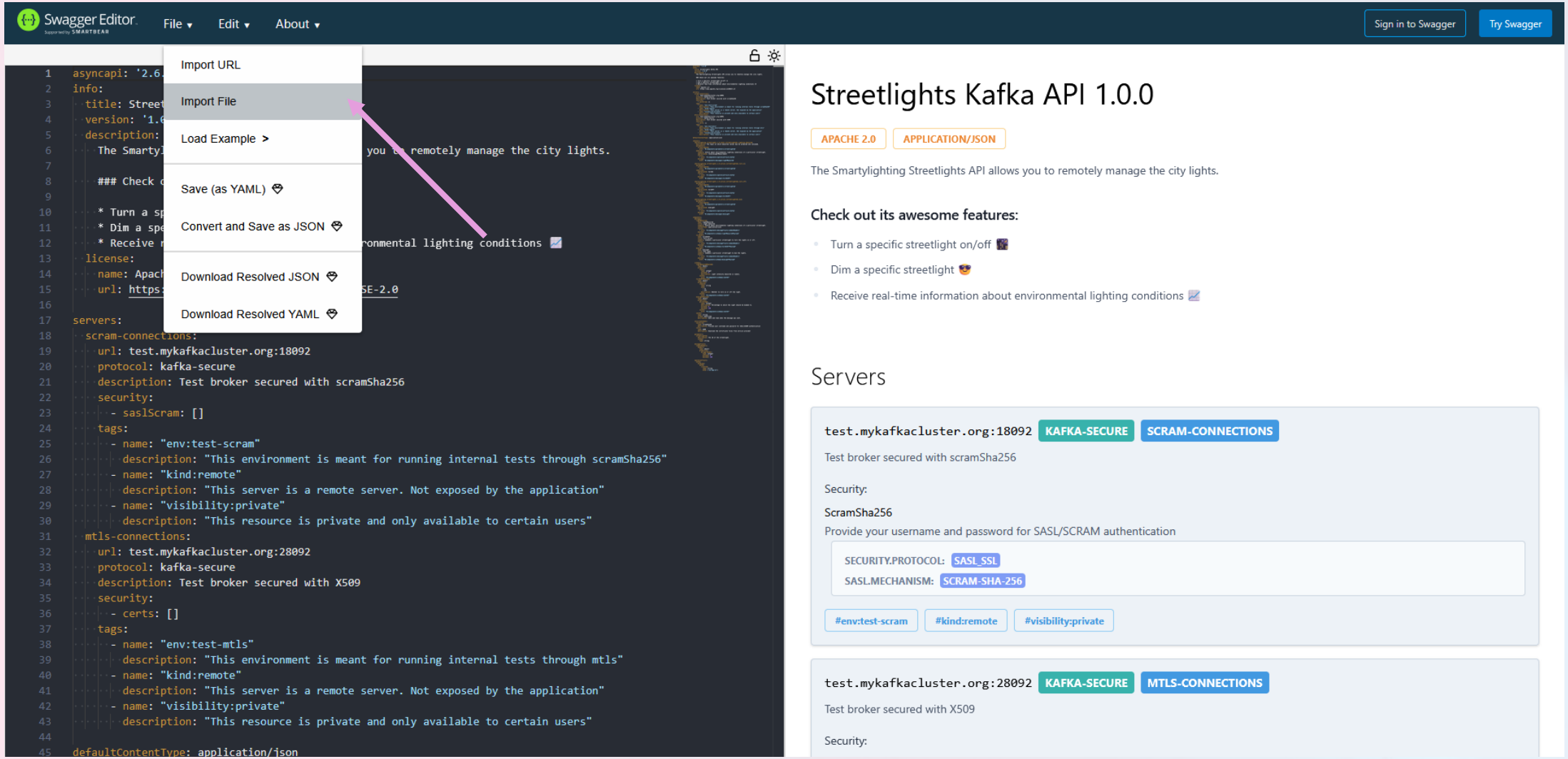
      responses:
        '201':
          description: Projeto criado

components:
  schemas:
    CreateProject:
      type: object
      required:
        - name
      properties:
        name:
          type: string
          minLength: 1
        description:
          type: string
```

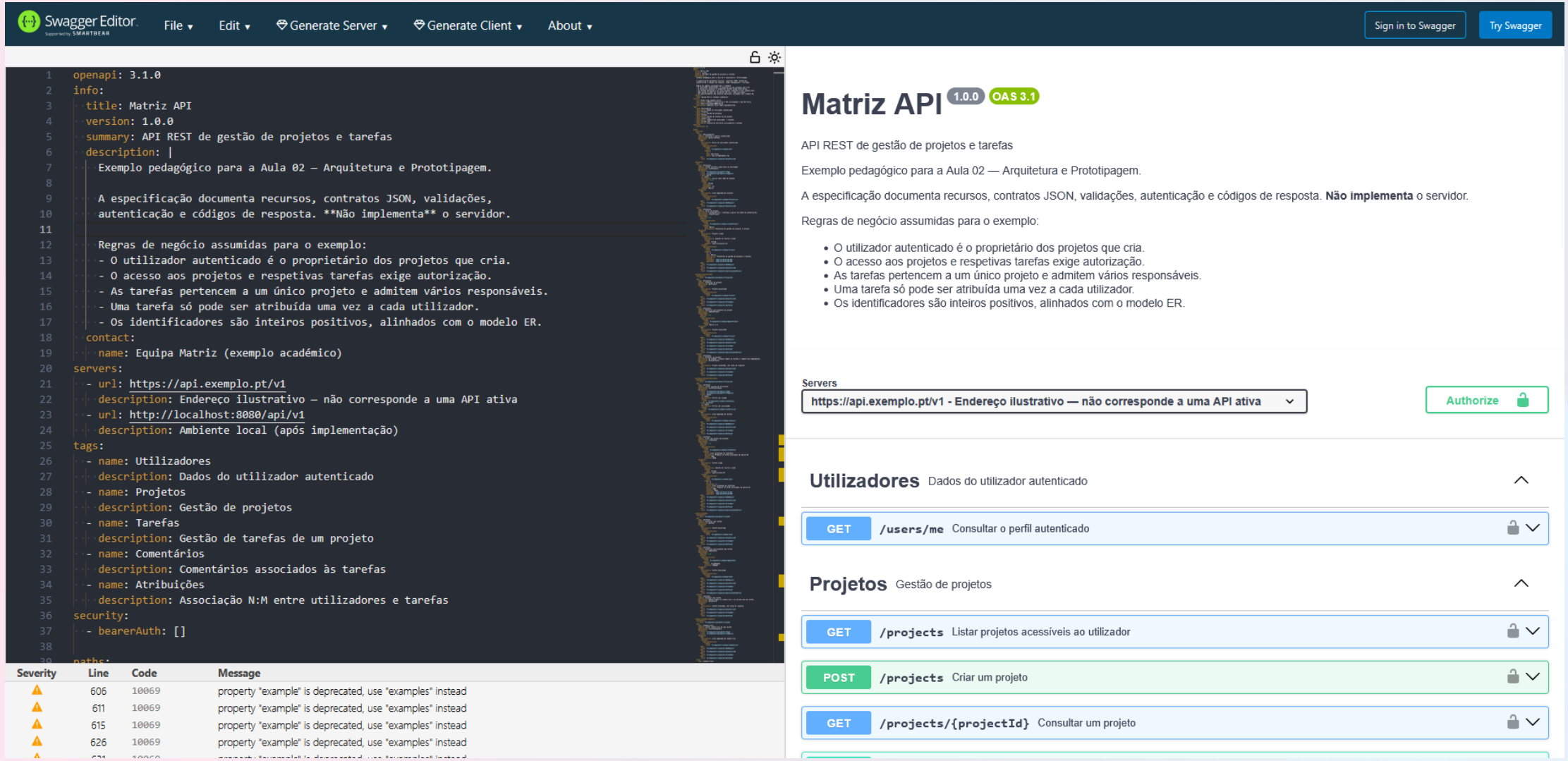
EXEMPLO — OpenAPI 3.1



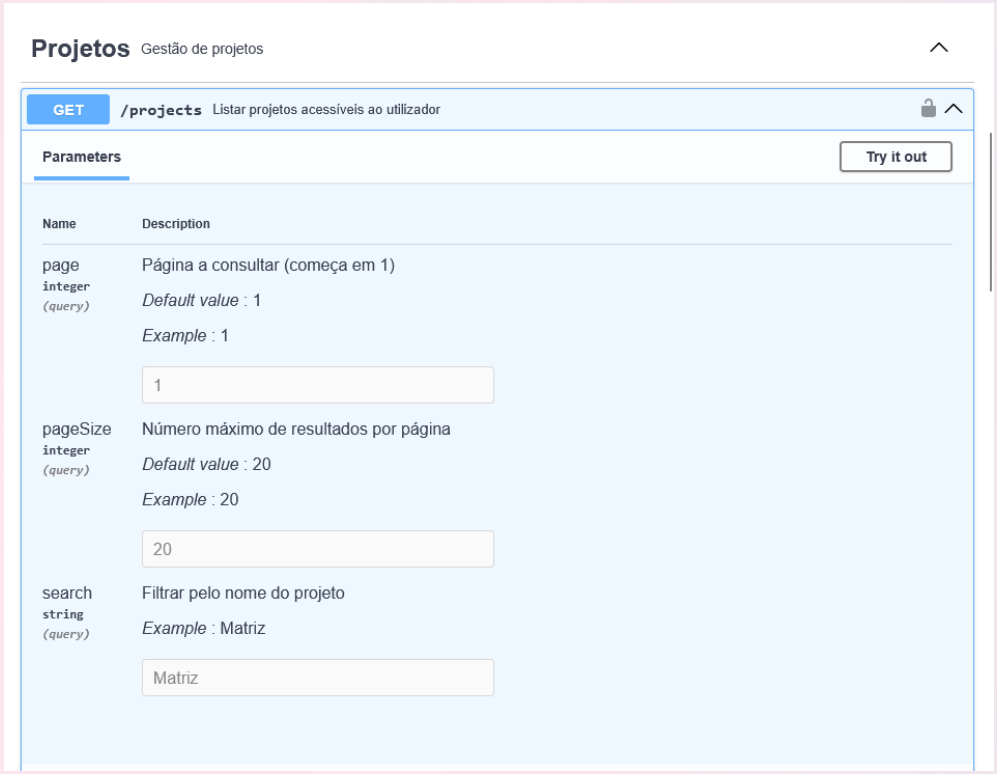
EXEMPLO — OpenAPI 3.1

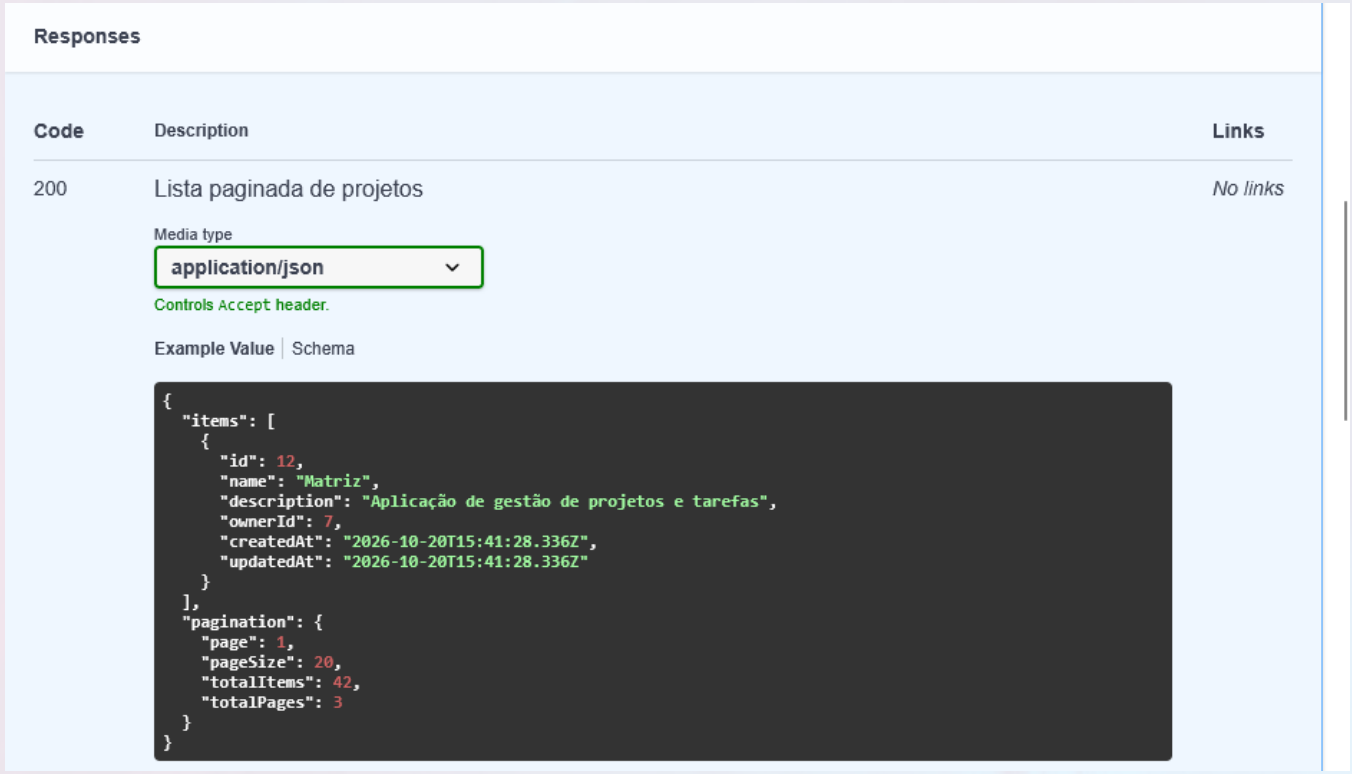


EXEMPLO — OpenAPI 3.1

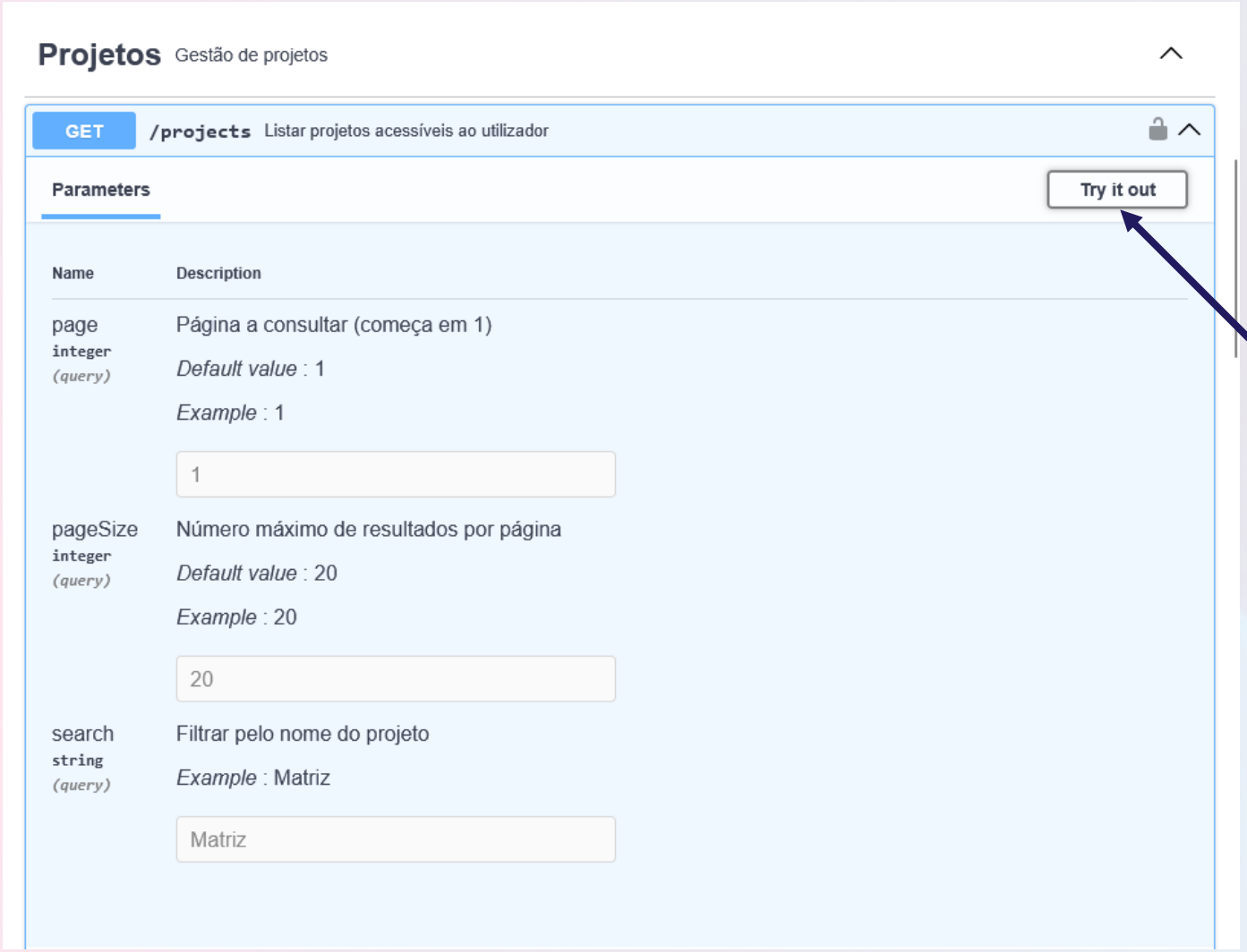


EXEMPLO — OpenAPI 3.1

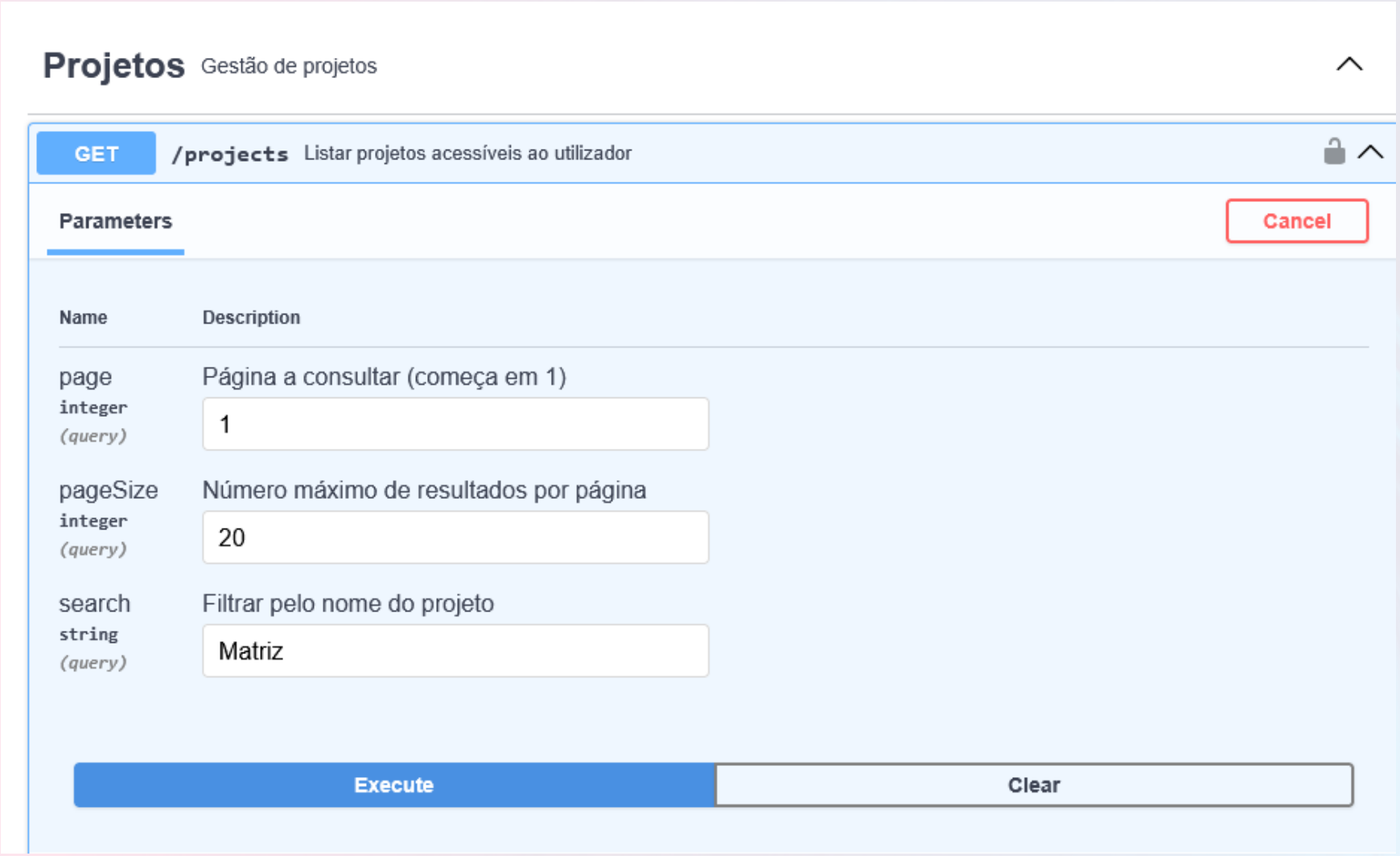




EXEMPLO — OpenAPI 3.1



EXEMPLO — OpenAPI 3.1



GERAR E VALIDAR CONTRATOS DE API COM IA

- Utilizar o contexto do projeto para desenhar uma API coerente.
- Contexto necessário:
  - Requisitos e User Stories.
  - Modelo de dados aprovado.
  - Arquitetura e decisões técnicas.
  - Restrições de segurança e privacidade.
  - Convenções e instruções do projeto.
  - Consultar exemplo de prompt
- Pode consultar o exemplo de prompt usado para gerar documentação, na pasta **06 - Anexos**

GERAR E VALIDAR CONTRATOS DE API COM IA

- A geração de código deve respeitar os contratos aprovados, não redefini-los arbitrariamente.



Requirements → API Design → OpenAPI → Review → Human Approval

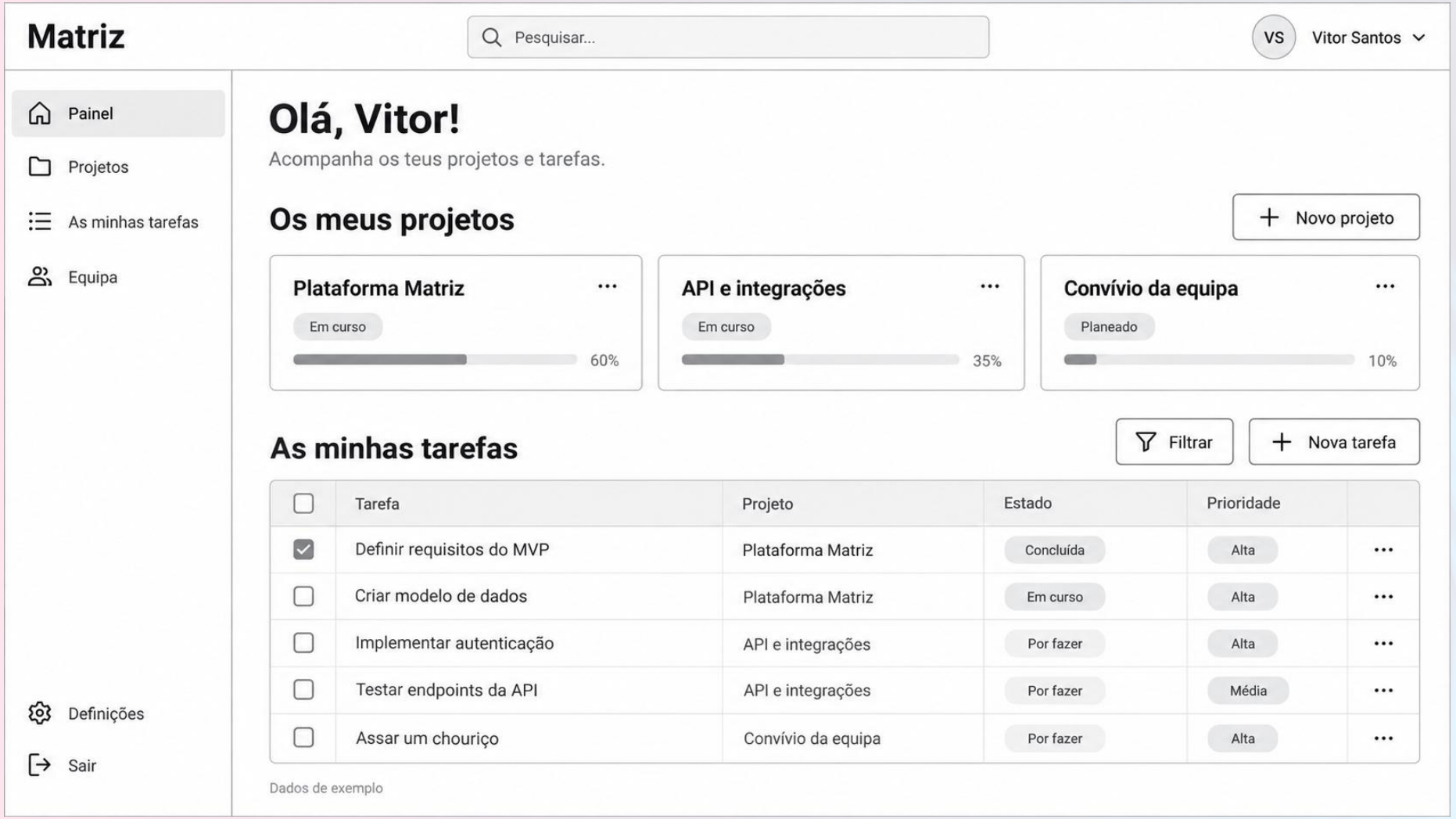
PROTOTIPAGEM DE INTERFACES

VITOR MANUEL SANTOS

PROTOTIPAGEM DE INTERFACES COM IA

- A IA permite acelerar a criação de interfaces através de:
  - Geração de wireframes e layouts.
  - Criação de protótipos de alta fidelidade.
  - Transformação de descrições em interfaces.
  - Conversão de imagens e screenshots em código.
  - Geração de componentes reutilizáveis.
  - Exploração de alternativas de UI/UX.
- Uma interface visualmente apelativa não garante uma boa experiência de utilização.

AI-ASSISTED UI DESIGN

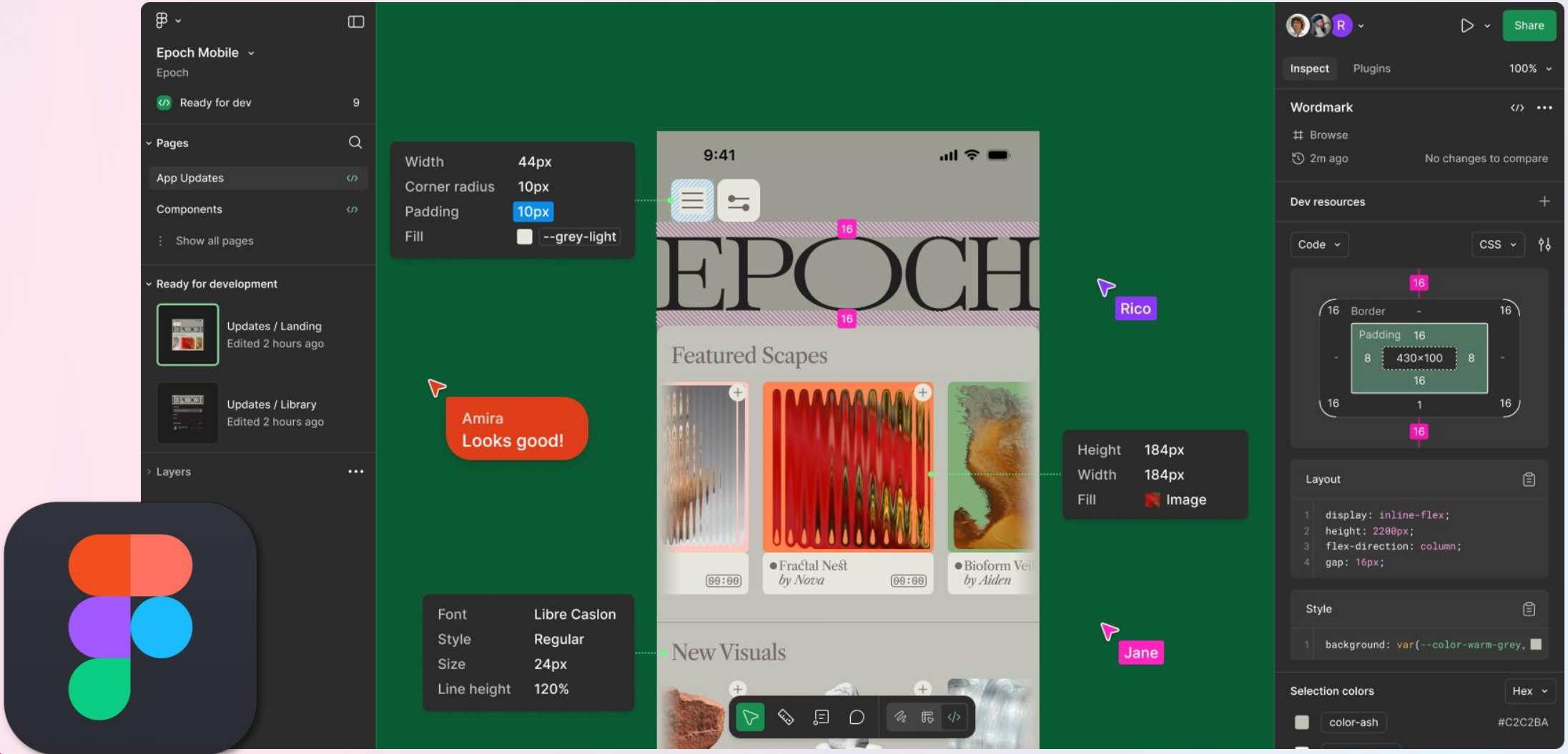


FERRAMENTAS DE PROTOTIPAGEM

- Figma — Design e prototipagem.
- v0 — Geração de interfaces e componentes.
- Lovable — Prototipagem de aplicações.
- Coding Agents — Geração e integração de UI no projeto.

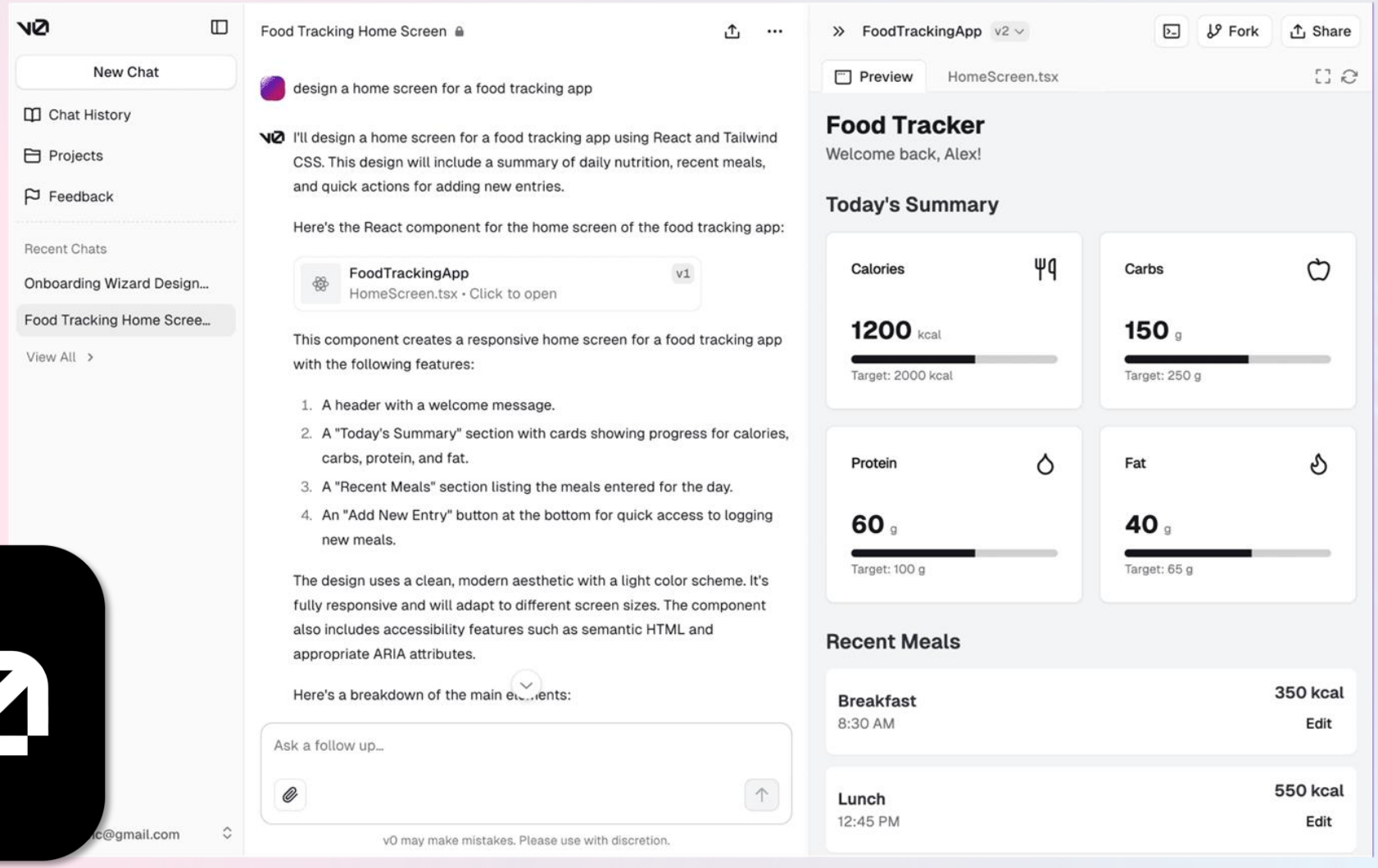
- Todas estas ferramentas integram AI Assistants internos que poderão ser utilizados para agilizar a tarefas de construção e desenho de wireframes e protótipos.

FERRAMENTAS DE PROTOTIPAGEM – FIGMA

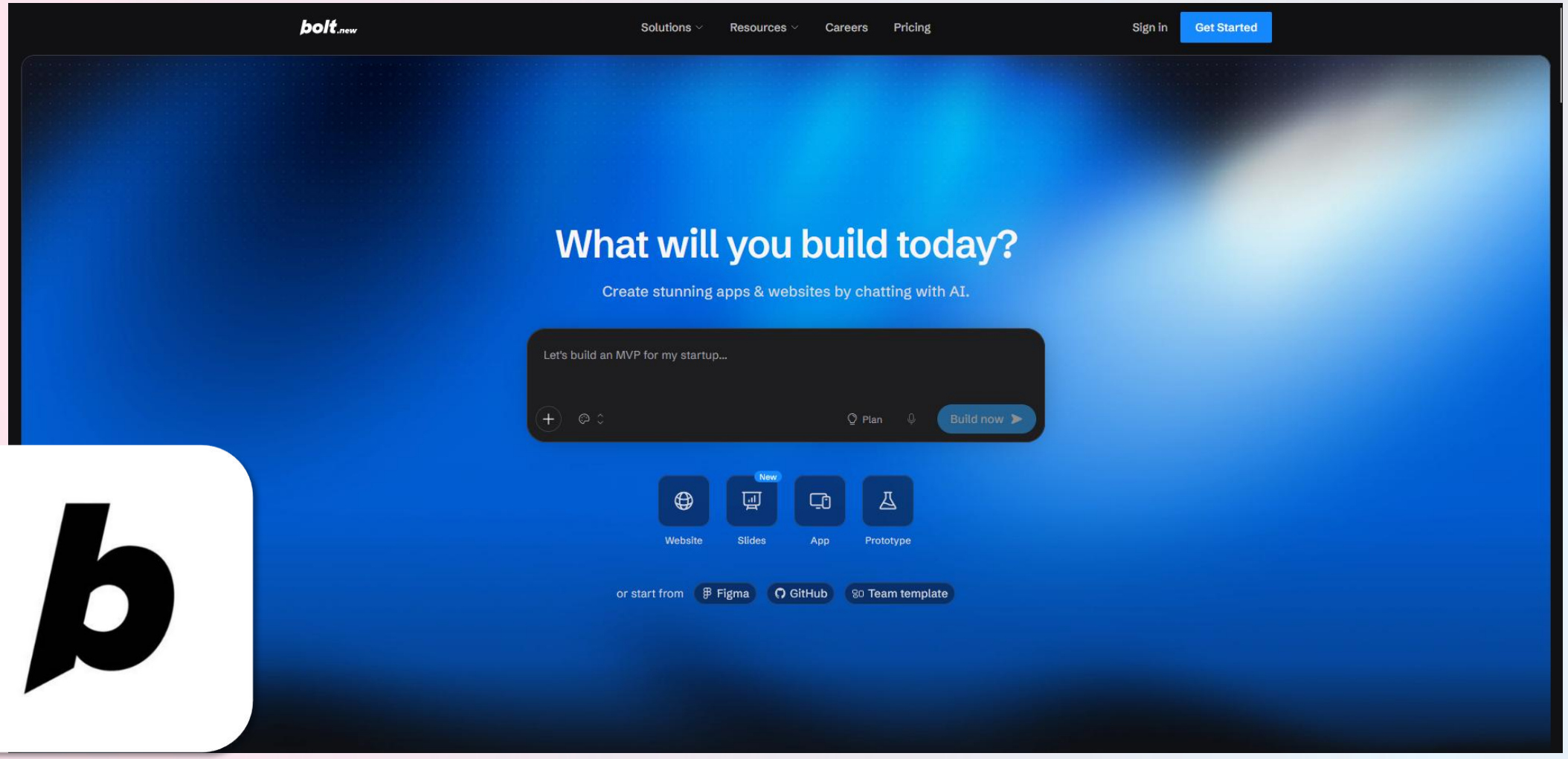


VITOR MANUEL SANTOS

FERRAMENTAS DE PROTOTIPAGEM – V0

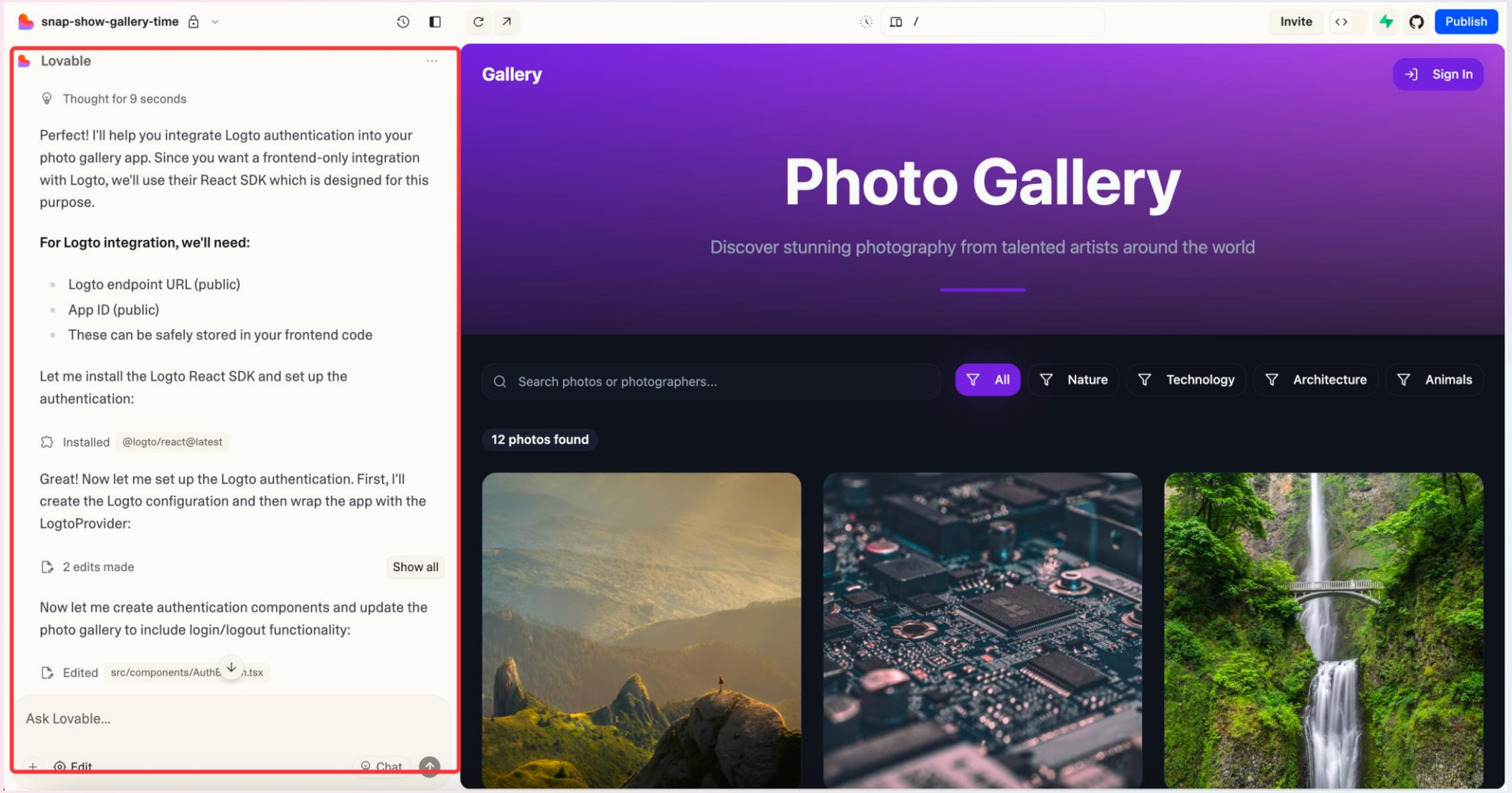


FERRAMENTAS DE PROTOTIPAGEM – BOLT.NEW



VITOR MANUEL SANTOS

FERRAMENTAS DE PROTOTIPAGEM – Lovable



PROMPT ENGINEERING PARA PROTOTIPAGEM

- O resultado depende do contexto e das restrições fornecidas.
- Contexto necessário:
  - Requisitos funcionais e User Stories.
  - Tipos de utilizador e principais workflows.
  - Arquitetura e tecnologias aprovadas.
  - Contratos de API, quando disponíveis.
  - Design System e identidade visual, se existentes.

COMPONENTIZAÇÃO, RESPONSIVIDADE E ACESSIBILIDADE

Dimensão	Boas práticas
Componentização	Componentes reutilizáveis e responsabilidades claras
Design System	Cores, tipografia, espaçamento e padrões consistentes
Responsividade	Adaptação a diferentes ecrãs
Acessibilidade	HTML semântico, navegação por teclado e WCAG
Gestão de estados	Loading, Empty, Error e Success
Manutenção	Separação entre apresentação, estado e lógica de negócio

EXEMPLO DE ESTRUTURA DE COMPONENTES

```
src/
├── components/
│   ├── ui/
│   ├── layout/
│   └── common/
├── features/
│   ├── projects/
│   └── tasks/
└── pages/
```

VALIDAÇÃO DE PROTÓTIPOS GERADOS POR IA

- O resultado depende do contexto e das restrições fornecidas.
- Contexto necessário:
  - Requisitos funcionais e User Stories.
  - Tipos de utilizador e principais workflows.
  - Arquitetura e tecnologias aprovadas.
  - Contratos de API, quando disponíveis.
  - Design System e identidade visual, se existentes.

REFERÊNCIAS

- https://swagger.io/open-source/swagger-ui/
- https://editor.swagger.io/
- https://www.postman.com/
- https://swagger.io/blog/swagger-editor-a-new-era-begins/